RKNN3 算子支持与约束参考

文件标识：RK-YH-YF-418

发布版本：v1.2.0

日期：2026-10-23

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

RKNN3 算子支持与约束参考

实现路径标签：`Subgraph`、`RegEmit`、`NRV`、`NRVReg`、`Runtime/Simulator fallback`。实际路径由编译器根据目标平台、shape、数据类型和算子参数分发；标签不替代各算子的支持范围与约束。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	修改人	修改日期	修改内容
v1.2.0	NPU 团队	2026-10-23	按当前源码重新汇总 RK182X/RK3572 算子规格、数据类型与参数约束；补充 Atan、Tan、exPagedAttention，并纳入 Cast、exDataConvert 类型转换算子。
v1.1.0	NPU 团队	2026-08-14	1. 初次发布 RK18X/RK3572 平台算子支持与约束参考

目录

RKNN3 算子支持与约束参考

RK182X

卷积

Conv

ConvTranspose

ConvActivation

ConvRelu

ConvClip

ConvSigmoid

ConvTanh

ConvElu

ConvHardSigmoid

ConvHardSwish

ConvSwish

ConvSoftplus

ConvMish

ConvAdd

ConvAddRelu

ConvPRelu

ConvTransposeActivation

ConvTransposeRelu

ConvTransposeClip

ConvTransposeSigmoid

ConvTransposeTanh

ConvTransposeElu

ConvTransposeHardSigmoid

ConvTransposeHardSwish

- ConvTransposeSwish
- ConvTransposeSoftplus
- ConvTransposeMish
- ConvTransposeAdd
- ConvTransposePRelu
- exConvStreaming

池化

- AveragePool
- MaxPool
- GlobalAveragePool
- GlobalMaxPool

元素级运算

- Add
- Sub
- Mul
- Div
- Min
- Max
- Pow
- Neg
- Exp
- Sqrt
- Reciprocal
- Floor
- Round
- Sin
- Cos
- Atan
- Tan
- exFastExp
- exTwoPow
- AddRelu
- MulRelu

激活函数

- Relu
- Sigmoid

- Clip
- LeakyRelu
- PRelu
- Elu
- Softplus
- HardSigmoid
- HardSwish
- Tanh
- Mish
- exGelu
- exHardSwish
- exMish
- exSwish
- exGlu
- exLogit

比较与逻辑运算

- Equal
- Greater
- GreaterOrEqual
- Less
- LessOrEqual
- Not
- And
- Where

归约运算

- ReduceMax
- ReduceMean
- ReduceSum
- ArgMax
- ArgMin
- TopK

张量变换

- Reshape
- Transpose
- Concat
- Split

Expand

Slice

Pad

DepthToSpace

SpaceToDepth

Resize

exWindow

索引与散布

Gather

GatherElements

GatherND

ScatterND

归一化

BatchNormalization

BatchNormRelu

exNorm

exSoftmax

exSoftmaxMask

矩阵运算

exMatMul

exMatMulActivation

LLM 扩展算子

exAttention

exPagedAttention

exSDPAttention

exLinearAttention

exRoPE

exSum

类型转换

Cast

exDataConvert

输入输出处理

exInputProc

exOutputProc

RK3572

卷积

- Conv
- ConvTranspose
- ConvActivation
- ConvRelu
- ConvClip
- ConvSigmoid
- ConvTanh
- ConvElu
- ConvHardSigmoid
- ConvHardSwish
- ConvSwish
- ConvSoftplus
- ConvMish
- ConvAdd
- ConvAddRelu
- ConvPRelu
- ConvTransposeActivation
- ConvTransposeRelu
- ConvTransposeClip
- ConvTransposeSigmoid
- ConvTransposeTanh
- ConvTransposeElu
- ConvTransposeHardSigmoid
- ConvTransposeHardSwish
- ConvTransposeSwish
- ConvTransposeSoftplus
- ConvTransposeMish
- ConvTransposeAdd
- ConvTransposePRelu
- exConvStreaming

池化

- AveragePool
- MaxPool
- GlobalAveragePool
- GlobalMaxPool

元素级运算

- Add
- Sub
- Mul
- Div
- Min
- Max
- Pow
- Neg
- Exp
- Sqrt
- Reciprocal
- Floor
- Round
- Sin
- Cos
- Atan
- Tan
- exFastExp
- exTwoPow
- AddRelu
- MulRelu

激活函数

- Relu
- Sigmoid
- Clip
- LeakyRelu
- PRelu
- Elu
- Softplus
- HardSigmoid
- HardSwish
- Tanh
- Mish
- exGelu
- exHardSwish
- exMish
- exSwish

- exGlu
- exLogit

比较与逻辑运算

- Equal
- Greater
- GreaterOrEqual
- Less
- LessOrEqual
- Not
- And
- Where

归约运算

- ReduceMax
- ReduceMean
- ReduceSum
- ArgMax
- ArgMin
- TopK

张量变换

- Reshape
- Transpose
- Concat
- Split
- Expand
- Slice
- Pad
- DepthToSpace
- SpaceToDepth
- Resize
- exWindow

索引与散布

- Gather
- GatherElements
- GatherND
- ScatterND

归一化

BatchNormalization
BatchNormRelu
exNorm
exSoftmax
exSoftmaxMask
矩阵运算
exMatMul
exMatMulActivation
LLM 扩展算子
exAttention
exPagedAttention
exSDPAttention
exLinearAttention
exRoPE
exSum
类型转换
Cast
exDataConvert
输入输出处理
exInputProc
exOutputProc

RK182X

卷积

Conv

卷积计算。

公式

`output = Conv(input_tensor, weight, bias)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::_convolution`

输入

- **input_tensor** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **weight** - `T1` (必选):
  - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
  - 量化: per-layer, per-channel
  - kernel_h 范围: [1, 31]
  - kernel_w 范围: [1, 31]
- **bias** - `T1` (可选):
  - Shape: C[output_channel]
  - 量化: per-layer

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **strides - int[stride_h, stride_w]**:

    - stride_h 范围: [1, 7]
    - stride_w 范围: [1, 7]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
    - pads_top 范围: [0, 15]
    - pads_left 范围: [0, 15]
    - pads_bottom 范围: [0, 15]
    - pads_right 范围: [0, 15]
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h 范围: [1, 31]
    - dilation_w 范围: [1, 31]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

特殊情况

Depthwise Convolution 当 Conv 按 depthwise 卷积分组语义使用时，适用以下差异规格。

- **输入差异** — **weight**：kernel_h 范围: [1, 8]；kernel_w 范围: [1, 8]
- **属性差异** — **dilations**：dilation_h 范围: [1, 32]；dilation_w 范围: [1, 32]

ConvTranspose

转置卷积计算。

公式

```
output = ConvTranspose(input_tensor, weight, bias)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::conv_transpose1d` , `aten::conv_transpose2d`

输入

- **input_tensor** - `T1` (必选):
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

    - 当 dilated_kernel_h > 1 时，input_tensor.width < 16383。
      首层输入 width 还需满足模型输入限制。
- **weight** - `T1` (必选):
    - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
    - 量化: per-layer, per-channel
    - kernel_h 范围: [1, 31]
    - kernel_w 范围: [1, 31]
- **bias** - `T1` (可选):
    - Shape: C[output_channel]
    - 量化: per-layer

输出

- **output** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

属性

- **strides - int[stride_h, stride_w]**:
    - stride_h 范围: [1, 8]
    - stride_w 范围: [1, 8]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
    - pads_top 范围: [0, 15]
    - pads_left 范围: [0, 15]
    - pads_bottom 范围: [0, 15]
    - pads_right 范围: [0, 15]
    - 不支持 kernel_h * dilation_h - dilation_h - pads_top < 0
      不支持 kernel_w * dilation_w - dilation_w - pads_left < 0
      不支持 stride_h * (height - 1) - pads_top + 1 < output_h
      不支持 stride_w * (width - 1) - pads_left + 1 < output_w
- **group - int**:
    - 支持 group = 1；当 num_input = num_output 时，也支持 group = num_output。
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h 范围: [1, 32]
    - dilation_w 范围: [1, 32]
    - 当 dilation != (1, 1) 时，仅支持 kernel = (1, 1)。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

ConvActivation

Conv 与通用 Activation 融合计算，激活类型由 Activation 子算子的类型决定。

公式

```
output = Activation(Conv(input_tensor, weight, bias))
```

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

```
aten::_convolution
```

ConvRelu

Conv 与 Relu 融合计算。

公式

```
output = Relu(Conv(input_tensor, weight, bias))
```

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

```
aten::_convolution
```

ConvClip

Conv 与 Clip 融合计算。

公式

`output = Clip(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvSigmoid

Conv 与 Sigmoid 融合计算。

公式

`output = Sigmoid(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvTanh

Conv 与 Tanh 融合计算。

公式

`output = Tanh(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvElu

Conv 与 Elu 融合计算。

公式

`output = Elu(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvHardSigmoid

Conv 与 HardSigmoid 融合计算。

公式

`output = HardSigmoid(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvHardSwish

Conv 与 HardSwish 融合计算。

公式

`output = HardSwish(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvSwish

Conv 与 Swish 融合计算。

公式

`output = Swish(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvSoftplus

Conv 与 Softplus 融合计算。

公式

`output = Softplus(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvMish

Conv 与 Mish 融合计算。

公式

`output = Mish(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvAdd

Conv 与 Add 融合计算。

公式

`output = Add(Conv(input_tensor, weight, bias), residual)`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

额外输入

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- Add 的第二输入。
  支持广播，广播约束同 Add。

ConvAddRelu

Conv 与 Add 及 Relu 融合计算。

公式

`output = Relu(Add(Conv(input_tensor, weight, bias), residual))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

额外输入

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - Add 的第二输入。
    支持广播，广播约束同 Add。

ConvPRelu

Conv 与 PRelu 融合计算。

公式

`output = PRelu(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvTransposeActivation

ConvTranspose 与通用 Activation 融合计算，激活类型由 Activation 子算子的类型决定。

公式

```
output = Activation(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeRelu

ConvTranspose 与 Relu 融合计算。

公式

```
output = Relu(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeClip

ConvTranspose 与 Clip 融合计算。

公式

`output = Clip(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeSigmoid

ConvTranspose 与 Sigmoid 融合计算。

公式

`output = Sigmoid(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeTanh

ConvTranspose 与 Tanh 融合计算。

公式

`output = Tanh(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeElu

ConvTranspose 与 Elu 融合计算。

公式

```
output = Elu(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeHardSigmoid

ConvTranspose 与 HardSigmoid 融合计算。

公式

```
output = HardSigmoid(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeHardSwish

ConvTranspose 与 HardSwish 融合计算。

公式

`output = HardSwish(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeSwish

ConvTranspose 与 Swish 融合计算。

公式

`output = Swish(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeSoftplus

ConvTranspose 与 Softplus 融合计算。

公式

`output = Softplus(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeMish

ConvTranspose 与 Mish 融合计算。

公式

`output = Mish(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeAdd

ConvTranspose 与 Add 融合计算。

公式

`output = Add(ConvTranspose(input_tensor, weight, bias), residual)`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

额外输入

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- Add 的第二输入。
  支持广播，广播约束同 Add。

ConvTransposePRelu

ConvTranspose 与 PRelu 融合计算。

公式

`output = PRelu(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

---

exConvStreaming

带流式 cache 的卷积计算。

公式

`y = Conv(Concat(cache, x), w, b)`
`cache = Slice(Concat(cache, Slice(x, width=[0, tc])), width=[end - cache_w, end])`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::_convolution`

输入

- **x** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **w** - `T1` (必选):

    - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
    - 量化: per-layer, per-channel
    - kernel_h 范围: [1, 31]
    - kernel_w 范围: [1, 31]
- **b** - `T1` (可选):
    - Shape: C[output_channel]
    - 量化: per-layer
    - 可缺省；无 bias 且需要传 tc 时，b 可为空输入占位。
- **tc** - `T2` (可选):
    - Shape: N[1]
    - tc 表示本次写入 cache 的有效宽度，长度需小于输入 width。
      输入个数为 3 时，第三个输入若尺寸等于 weight 的 output_channel，则按 b 处理，否则按 tc 处理。

输出

- **y** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

属性

- **strides - int[stride_h, stride_w]**:
    - stride_h 范围: [1, 7]
    - stride_w 范围: [1, 7]
- **pads - int[pads_top, cache_w, pads_bottom, pads_right]**:
    - pads_top 范围: [0, 15]
    - pads_bottom 范围: [0, 15]
    - pads_right 范围: [0, 15]
    - cache_w 为流式 cache 长度，必须大于 0。
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h 范围: [1, 31]
    - dilation_w 范围: [1, 31]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

特殊情况

Depthwise exConvStreaming 当 exConvStreaming 按 depthwise 卷积分组语义使用时，适用以下差异规格。

- **输入差异** — **w**：kernel_h 范围: [1, 8]；kernel_w 范围: [1, 8]
- **属性差异** — **dilations**：dilation_h 范围: [1, 32]；dilation_w 范围: [1, 32]

说明

子图算子，实际实现由 createSubGraph 在 IR 层展开，emit 仅返回成功。

池化

AveragePool

按指定 kernel_shape、strides 和 pads 对输入 Tensor 做平均池化。

公式

```
Y = average_pool(X, kernel_shape, strides, pads)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::adaptive_avg_pool2d`, `aten::avg_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8/int16 量化需满足 height <= 32 且 width <= 32

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **kernel_shape - int[kernel_h, kernel_w]**:

    - kernel_h 范围: [1, 32]
    - kernel_w 范围: [1, 32]
- **strides - int[stride_h, stride_w]**:
    - stride_h 范围: [1, 63]
    - stride_w 范围: [1, 63]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
    - pad_top 范围: [0, 63]
    - pad_left 范围: [0, 63]
    - pad_bottom 范围: [0, 63]
    - pad_right 范围: [0, 63]
- **auto_pad - string**:
    - 仅支持显式 pads。
- **ceil_mode - int**:
    - ceil 模式通过补充 bottom/right padding 支持。
- **count_include_pad - int**:
    - 为 1 时均值计算包含 padding 区域。
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h 范围: [1, 1]
    - dilation_w 范围: [1, 1]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

MaxPool

按指定 kernel_shape、strides 和 pads 对输入 Tensor 做最大池化。

公式

`Y = max_pool(X, kernel_shape, strides, pads)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::max_pool1d` , `aten::max_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **kernel_shape - int[kernel_h, kernel_w]**:
  - kernel_h 范围: [1, 32]
  - kernel_w 范围: [1, 256]
  - kernel_h * kernel_w <= 256
- **strides - int[stride_h, stride_w]**:
  - stride_h 范围: [1, 31]
  - stride_w 范围: [1, 256]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
  - pad_top 范围: [0, 63]
  - pad_left 范围: [0, 63]
  - pad_bottom 范围: [0, 63]
  - pad_right 范围: [0, 63]
- **auto_pad - string**:
  - 仅支持显式 pads。
- **ceil_mode - int**:
  - ceil 模式通过补充 bottom/right padding 支持。
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h 范围: [1, 1]
  - dilation_w 范围: [1, 1]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

GlobalAveragePool

对输入 Tensor 的 H/W 空间维度做全局平均池化。

公式

`Y = mean(X, axes=[2, 3])`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::adaptive_avg_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8 量化需满足 height <= 32 且 width <= 32；

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, 1, 1]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

GlobalMaxPool

对输入 Tensor 的空间维度做全局最大池化，输出每个通道的最大值。

公式

`Y[batch, channel, 0, 0] = max(X[batch, channel, :, :])`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::adaptive_max_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 硬约束：height * width <= 256 且 height <= 32。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, 1, 1]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

元素级运算

Add

执行元素级二进制加法，支持 Numpy 样式广播。

公式

`C = A + B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::add` , `aten::add_`

输入

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<int32>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Sub

执行元素级二进制减法，支持 Numpy 样式广播。

公式

`C = A - B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sub` , `aten::sub_`

输入

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Mul

执行元素级二进制乘法，支持 Numpy 样式广播。

公式

`C = A * B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::mul` , `aten::mul_`

输入

- **A** - `T1, T2` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Div

执行元素级二进制除法，支持 Numpy 样式广播。

公式

`C = A / B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::div`, `aten::true_divide`

输入

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`const_tensor<float16>`):

Min

对多个输入张量逐元素求最小值，支持 Numpy 样式广播。

公式

`C = min(A, B, ...)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::min`, `aten::minimum`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer, per-channel

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Max

对多个输入张量逐元素求最大值，支持 Numpy 样式广播。

公式

`C = max(A, B, ...)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::max`, `aten::maximum`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

    - 量化: per-layer
    - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer
    - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer, per-channel

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Pow

执行元素级指数运算。

公式

`output = input ^ exponent`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::pow` , `aten::pow_`

输入

- **input** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer
- **exponent** - `T2`:
    - Shape: S[scalar]

输出

- **output** - `T1`:

- Shape: NCHW[batch, channel, height, width]
- 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Neg

对输入张量执行元素级取负操作。

公式

`Y = -X`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::neg` , `aten::neg_`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Exp

对输入张量逐元素计算以 e 为底的指数。

公式

`output = exp(input)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::exp`, `aten::exp_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Sqrt

对输入张量执行元素级平方根运算。

公式

```
output = sqrt(input)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sqrt`, `aten::sqrt_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Reciprocal

对输入张量执行元素级倒数运算。

公式

```
Y = 1 / X
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::reciprocal` , `aten::reciprocal_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Floor

对输入张量逐元素向下取整（向负无穷方向取最接近的整数）。

公式

`output = floor(input)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::floor`, `aten::floor_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Round

取整函数，采用 round-half-up 语义，公式为 y = Floor(x + 0.5)。与 ONNX Round 的 round-half-to-even（银行家舍入）不同。

公式

`y = Floor(x + 0.5)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::round`, `aten::round_`

输入

- **x** - `T1`:

- Shape: NCHW[batch, channel, height, width]

输出

- **y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<float16>`):

Sin

按元素计算正弦函数。

公式

`y = sin(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sin` , `aten::sin_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

说明

实现先对输入做 mod 2π，再查 LUT 表。

Cos

按元素计算余弦函数。

公式

`y = cos(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::cos`, `aten::cos_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

说明

实现先对输入做 mod 2π，再查 LUT 表。

Atan

按元素计算反正切函数。

公式

`y = atan(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::atan`, `aten::atan_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

说明

LUT 主表覆盖 x ∈ [-16, 16]，该范围外按斜率 1/(1+x^2) 插值逼近，输出收敛于 ±π/2。

Tan

按元素计算正切函数。

公式

`y = tan(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::tan`, `aten::tan_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<float16>` , `tensor<bfloat16>`):

说明

实现先对输入做 mod π（结果区间 [-π/2, π/2]），再查 LUT 表。LUT 主表仅覆盖 [-1, 1]，该范围外输出钳位为 tan(±1)，因此大输入（mod π 后超出 [-1, 1]）无法保证精度。

---

exFastExp

按元素快速计算自然指数，近似实现以提升性能。

公式

`output ≈ exp(input)`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::exp` , `aten::exp_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exTwoPow

按元素计算 2 的幂次方。

公式

`y = 2^x`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::pow`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

AddRelu

Add 与 Relu 融合计算。

公式

`C = Relu(A + B)`

继承关系

继承自 Add。

算子层级

融合算子

PYTORCH 来源

`aten::add`, `aten::add_`

MulRelu

Mul 与 Relu 融合计算。

公式

`C = Relu(A * B)`

继承关系

继承自 Mul。

算子层级

融合算子

PYTORCH 来源

`aten::mul`, `aten::mul_`

激活函数

Relu

整流线性单元，对输入逐元素做 max(input, 0)。

公式

`output = max(input, 0)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::relu`, `aten::relu6`, `aten::relu_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Sigmoid

给定输入张量，按元素应用 Sigmoid 函数。

公式

```
y = 1 / (1 + exp(-x))
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sigmoid` , `aten::sigmoid_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Clip

将输入张量的元素裁剪到 [min, max] 区间，超出边界的值被限制到边界。

公式

`output = Clip(input, min, max)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::clamp`, `aten::clamp_`

输入

- **input** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **min** - `T2` (可选):
  - Shape: S[scalar]
  - 裁剪下界，标量。缺省表示不限制下界。
- **max** - `T2` (可选):
  - Shape: S[scalar]
  - 裁剪上界，标量。缺省表示不限制上界。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

LeakyRelu

Leaky 整流线性单元，正值保持不变，负值按固定比例 alpha 缩放。

公式

`output = input if input >= 0 else alpha * input`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::leaky_relu`, `aten::leaky_relu_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **alpha - float[alpha]**:
  - alpha 范围: [0, 1]
  - 负值缩放比例。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

PRelu

参数化整流线性单元，正值保持不变，负值按可学习参数 slope 缩放。

公式

```
output = input if input >= 0 else slope * input
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::prelu`, `aten::prelu_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

    - 量化: per-layer, per-channel
- **slope** - `T2` :
    - Shape: C[channel]
    - 量化: per-channel
    - 支持单个标量或 C 维度系数。

输出

- **output** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer, per-channel

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Elu

按元素应用 Elu 函数，alpha = 1。

公式

`f(x) = alpha * (exp(x) - 1) for x < 0; f(x) = x for x >= 0`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::elu` , `aten::elu_`

输入

- **input** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

输出

- **output** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Softplus

按元素应用 Softplus 函数。

公式

`y = ln(exp(x) + 1)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::softplus`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

HardSigmoid

按元素应用 HardSigmoid 函数，由 alpha、beta 属性控制。

公式

`y = max(0, min(1, alpha * x + beta))`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::hardsigmoid`, `aten::hardsigmoid_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **alpha - float**:
  - 默认 0.2（与 ONNX 标准一致）。
- **beta - float**:
  - 默认 0.5。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

HardSwish

按元素应用 HardSwish 函数，alpha = 1/6，beta = 0.5。

公式

```
y = x * max(0, min(1, alpha * x + beta))
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::hardswish`, `aten::hardswish_`

输入

- **X** - `T1`:

- Shape: NCHW[batch, channel, height, width]
- 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Tanh

计算给定输入张量按元素的双曲正切。

公式

`y = tanh(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::tanh` , `aten::tanh_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Mish

按元素应用 Mish 函数。

公式

`y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::mish`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exGelu

按元素应用 Gelu 函数。

公式

`y = x * Phi(x)，当 Phi 取 tanh 近似时 y = 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715 * x^3)))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::gelu`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exHardSwish

按元素应用 HardSwish 函数，alpha = 1/6，beta = 0.5。

公式

`y = x * max(0, min(1, alpha * x + beta))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::hardswish`, `aten::hardswish_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exMish

按元素应用 Mish 函数。

公式

```
y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::mish` , `aten::mish_`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exSwish

按元素应用 Swish 函数（也称 SiLU）。

公式

```
y = x * sigmoid(x)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::silu`, `aten::silu_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exGlu

将输入 Tensor 在指定 axis 上均分为两部分，并对第二部分做 sigmoid 门控后与第一部分相乘。

公式

```
a, b = np.split(X, 2, axis=axis)
Y = a * sigmoid(b)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::glu`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - axis 指定维度的长度必须为偶数。
    NPU 子图实现主要面向 axis = 1 的 channel 维度拆分。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, out_channel, height, width]
  - 量化: per-layer
  - 输出 shape 与输入 shape 保持一致，但 axis 指定维度变为输入的一半。
    当 axis = 1 时，out_channel = channel / 2。

属性

- **axis - int**:
  - 指定 GLU 拆分的维度，取值范围为 [-rank, rank - 1]。
    负轴会按 Numpy 语义转换为正轴。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exLogit

对输入张量执行 logit 激活函数计算。

公式

```
Y = log(X / (1 - X))
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::logit`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

比较与逻辑运算

Equal

逐元素比较两个张量是否相等，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = (A == B)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::eq` , `aten::eq_`

输入

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - bool 语义结果，RKNN 以 int8 内存布局存储。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Greater

按元素比较 A 是否大于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = (A > B)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::gt`, `aten::gt_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`tensor<bool>`, `tensor<int8>`):

GreaterOrEqual

逐元素比较 A 是否大于等于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = (A >= B)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::ge`, `aten::ge_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - bool 语义结果，RKNN 以 int8 内存布局存储。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Less

按元素比较 A 是否小于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

```
C = (A < B)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::lt` , `aten::lt_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- 支持 Numpy 样式广播。

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

LessOrEqual

按元素比较 A 是否小于等于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = (A <= B)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::le` , `aten::le_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Not

对输入张量执行元素级逻辑取反操作。

公式

`Y = !X`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::logical_not`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

输出

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<bool>`):

And

执行元素级逻辑与运算，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = A && B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::bitwise_and`, `aten::logical_and`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<bool>`, `tensor<int8>`):

Where

根据 condition 条件选择 x 或 y 中的元素，支持 Numpy 样式广播。

公式

```
output = select(condition, x, y)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::where`

输入

- **condition** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
- **x** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

    - 支持 Numpy 样式广播。
- **y** - `T2`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer
    - 支持 Numpy 样式广播。

输出

- **output** - `T2`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

数据类型约束

T1 in (`tensor<bool>`): **T2** in (`tensor<int8>` , `tensor<float16>`):

归约运算

ReduceMax

沿指定轴对输入张量求最大值，支持多轴与 keepdims。

公式

`output = ReduceMax(input, axes, keepdims)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::amax` , `aten::max`

输入

- **input** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

输出

- **output** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

- keepdims=1 时被缩减的轴大小为 1，keepdims=0 时该轴从输出中消失，实际输出 rank 依 axes 和 keepdims 而定。

属性

- **axes - int[]**:
  - 计算最大值的轴。
- **keepdims - int**:
  - 是否保留缩减维度（1 保留，0 不保留）。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

ReduceMean

按指定 axes 对输入 Tensor 做均值归约。

公式

`Y = mean(data, axes)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::mean`

输入

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8/int16 量化且 axes = [2]、[3]、[2, 3]、[1, 2, 3] 时需满足 height <= 32 且 width <= 32
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - 指定需要归约的维度， 负轴会按 Numpy 语义转换为正轴。
    支持 axes = [1]、[2]、[3]、[2, 3]、[1, 2, 3]

输出

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ReduceSum

按指定 axes 对输入 Tensor 做求和归约。

公式

`Y = sum(data, axes)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sum`

输入

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8/int16 量化且 axes = [2]、[3]、[2, 3]、[1, 2, 3] 时需满足 height <= 32 且 width <= 32
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - 指定需要归约的维度， 负轴会按 Numpy 语义转换为正轴。
    支持 axes = [1]、[2]、[3]、[2, 3]、[1, 2, 3]

输出

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ArgMax

沿指定轴返回最大值的索引。

公式

`output = ArgMax(input, axis)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::argmax`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 指定在哪个轴上进行 argmax 操作，取值范围 [0, ndim-1]。
- **keepdims - int**:
  - 是否保持输出维度与输入相同。0 表示不保持，1 表示保持。
- **select_last_index - int**:
  - 当有多个最大值时，是否选择最后一个索引。0 表示选择第一个，1 表示选择最后一个。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ArgMin

沿指定轴返回最小值的索引。

公式

`output = ArgMin(input, axis)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::argmin`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 指定在哪个轴上进行 argmin 操作，取值范围 [0, ndim-1]。
- **keepdims - int**:
  - 是否保持输出维度与输入相同。0 表示不保持，1 表示保持。
- **select_last_index - int**:
  - 当有多个最小值时，是否选择最后一个索引。0 表示选择第一个，1 表示选择最后一个。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

TopK

沿指定轴检索输入张量中最大（或最小，由 largest 决定）的 K 个值及其索引。

公式

`(values, indices) = TopK(X, K, axis, largest, sorted)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::topk`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **K** - `T3` :
  - Shape: S[scalar]
  - 取最大的 K 个值，标量，存储在 int64 张量中。

输出

- **values** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 最大的 K 个值。
- **indices** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - 对应值的索引。ONNX 标准为 int64，RKNN 实际输出 int32 或 int64。

属性

- **axis - int**:
  - 计算的轴，默认 -1。
- **largest - int**:
  - 1 取最大，0 取最小，默认 1。
- **sorted - int**:
  - 1 按降序输出，0 不保证顺序，默认 1。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`): **T3** in (`tensor<int64>`):

张量变换

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

公式

`output = reshape(input, shape)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::reshape`, `aten::reshape_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **shape - int**:
  - 指定输出张量的形状。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Transpose

对输入张量按 perm 指定的轴顺序进行转置。

公式

`Y = transpose(X, perm)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::permute`, `aten::transpose`, `aten::transpose_`

输入

- **X** - `T1, T2`:
  - Shape: NCHW[n1, c1, h1, w1]
  - 量化: per-layer
  - 详细约束规则见属性列表 perm。

输出

- **Y** - `T1, T2`:
  - Shape: NCHW[n2, c2, h2, w2]
  - 量化: per-layer
  - 详细约束规则见属性列表 perm。

属性

- **perm - int[d0, d1, d2, d3]**:
  - 转置的轴顺序。不同 perm 参数的约束见下方 perm 约束表。

数据类型约束

T1 in (`tensor<int8>`): **T2** in (`tensor<int16>`, `tensor<float16>`):

说明

不同 perm 参数限制如下（其中 p = 268435456，q = 65536）：

perm	T1 约束	T2 约束
[0,1,2,3]	无限制	无限制
[0,1,3,2]	h1%16=0, h1*w1<16*p, h1*w1<2*q	h1%8=0, h1*w1<8*p, h1*w1<2*q
[0,2,3,1]	c1<8192, h1*w1%8=0, w1*c1<p*16	c1<8192, h1*w1%8=0, w1*c1<p*8
[0,2,1,3]	h1%16=0, h1*w1<16*p, h1*w1<2*q, c1*w1<p	h1%8=0, h1*w1<8*p, h1*w1<2*q, c1*w1<p
[0,3,1,2]	h1*w1<p, w1<8192, h1*c1<p	h1*w1<p, w1<8192, h1*c1<p
[0,3,2,1]	h1%16=0, h1*w1<16*p, h1*w1<2*q, c1<p*16	h1%8=0, h1*w1<8*p, h1*w1<2*q, c1<p*8
[1,0,2,3]	h1*w1<16*p, n1<8192, h1*w1<p	h1*w1<8*p, n1<8192, h1*w1<p
[1,0,3,2]	h1%16=0, c1*h1*w1<16*p, h1*w1<2*q	h1%8=0, c1*h1*w1<8*p, h1*w1<2*q

perm	T1 约束	T2 约束
[1,2,0,3]	n1*w1*<*p*16, c1*h1*w1<p, w1%16=0	n1*w1*<*p*8, c1*h1*w1<p, w1%8=0
[1,2,3,0]	h1*w1*<*p, n1*<*p*16, c1*h1*w1<p*16, n1*w1<p*16	h1*w1*<*p, n1*<*p*8, c1*h1*w1<p*8, n1*w1<p*8
[1,3,0,2]	c1*w1*<*8192, n1*h1<p, w1%16=0	c1*w1*<*8192, n1*h1<p, w1%8=0
[1,3,2,0]	c1*h1*w1<p, n1*h1<p, w1<8192, w1%16=0	c1*h1*w1<p, n1*h1<p, w1<8192, w1%8=0
[2,1,0,3]	c1<8192, c1*h1*w1<p, n1*w1<p	c1<8192, c1*h1*w1<p, n1*w1<p
[2,1,3,0]	c1*h1*w1<p, n1*w1*<*p*16, c1<8192, n1*w1%4=0	c1*h1*w1<p, n1*w1*<*p*8, c1<8192, n1*w1%4=0
[2,0,3,1]	c1<8192, c1*h1*w1<p	c1<8192, c1*h1*w1<p
[2,0,1,3]	n1*c1*h1*w1*<*p, h1*w1%16=0	n1*c1*h1*w1*<*p, h1*w1%8=0
[2,3,1,0]	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%16=0	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%8=0
[2,3,0,1]	h1*w1*<*8192, n1*c1<p	h1*w1*<*8192, n1*c1<p
[3,0,2,1]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%8=0
[3,0,1,2]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%8=0
[3,2,0,1]	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p
[3,2,1,0]	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*16, h1%16=0	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*8, h1%8=0
[3,1,0,2]	h1*w1*<*p, c1*w1<8192, n1*h1<p	h1*w1*<*p, c1*w1<8192, n1*h1<p
[3,1,2,0]	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p

---

Concat

沿指定 axis 将多个张量拼接成一个张量，所有输入除 axis 维外大小必须相同。

公式

```
output = Concat(input_tensor_list, axis)
```

算子层级

标准ONNX算子

PYTORCH 来源

```
aten::cat
```

输入

- **input_tensor** - `T`:
  - Shape: NCHW[batch, channel, height, width]

    - 量化: per-layer
    - 输入为张量列表，各张量除 axis 维外大小必须相同。

输出

- **output** - `T`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

属性

- **axis - int[axis]**:
    - axis 范围: [-4, 3]
    - 拼接轴，RKNN 限制输入为 4D NCHW，支持负轴。

数据类型约束

T in (`tensor<int8>` , `tensor<float16>`):

Split

沿指定 axis 将输入张量切分成多个输出张量。

公式

```
outputs = Split(input_tensor, axis, num_outputs, split)
```

算子层级

标准ONNX算子

PYTORCH 来源

```
aten::split
```

输入

- **input_tensor** - `T`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

输出

- **output** - `T`:
    - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer
- 一或多个输出张量，数量由 num_outputs 决定，shape 按切分情况决定。

属性

- **axis - int[axis]**:
  - axis 范围: [-4, 3]
  - 切分轴，RKNN 限制输入为 4D NCHW，支持负轴。
- **num_outputs - int**: —
- **split - int[]**:
  - 指定切分后各输出在 axis 方向上的大小。

数据类型约束

T in (`tensor<int8>` , `tensor<float16>`):

Expand

将输入张量广播到目标形状。

公式

```
output = Expand(input, target_shape)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::expand` , `aten::expand_as`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Slice

按 numpy 切片规则获取输入张量的一个切片。

公式

```
output = Slice(data, starts, ends, axes, steps)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::narrow` , `aten::slice`

输入

- **data** - `T` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **starts** - `T2` (必选):
  - Shape: D[dim]
  - 各轴切分起始位置。
- **ends** - `T2` (必选):
  - Shape: D[dim]
  - 各轴切分终止位置。
- **axes** - `T2` (可选):
  - Shape: D[dim]
  - 选取切分的轴，支持 0~3 轴，支持同时多轴选择；缺省时按全部轴处理。
- **steps** - `T2` (可选):
  - Shape: D[dim]
  - 各轴切分步长。当 height*width==1 且 channel%subc==0 且 starts[1]%subc==0 且 ends[1]%subc==0 且 steps[1]<=subc 时，steps 可不为 1； 否则仅当 steps==[1,1,1,1] 时可通过 NPU 运行。

输出

- **output** - `T` :

- Shape: NCHW[batch, channel, height, width]
- 量化: per-layer

数据类型约束

T in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

Pad

按指定 pads 和模式对输入张量进行填充。

公式

```
output = Pad(data, pads, mode, constant_value)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::constant_pad_nd`

输入

- **data** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **pads** - `T2` (必选):
  - Shape: PADS[batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end, height_end, width_end]
- **constant_value** - `T1` (可选):
  - Shape: S[scalar]
  - 量化: per-layer
  - 与 data 同类型，仅 constant 模式生效，缺省为 0。

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **mode - string**:
  - constant 模式无范围限制； edge 模式仅支持 h/w 方向。

数据类型约束

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int64>`):

特殊情况

Reflect Pad reflect 模式下输入 shape 分量有额外范围限制。

- **输入差异 — data**：channel 范围: [1, 8192]；height 范围: [1, 8192]；width 范围: [1, 8176]

DepthToSpace

将输入 Tensor 的 channel 维度按 blocksize 重排回 H/W 空间维度。

公式

```
N, C, H, W = X.shape
b = blocksize
out_channel = C // (b * b)
Y = np.empty((N, out_channel, H * b, W * b), dtype=X.dtype)

for n in range(N):
    for c in range(out_channel):
        for h in range(H * b):
            for w in range(W * b):
                offset = (h % b) * b + (w % b)
                in_c = c + offset * out_channel
                Y[n, c, h, w] = X[n, in_c, h // b, w // b]
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::pixel_shuffle`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - channel 必须能被 blocksize * blocksize 整除。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - 量化: per-layer
  - out_channel = channel / (blocksize * blocksize)。
    out_height = height * blocksize。
    out_width = width * blocksize。

属性

- **blocksize - int**:
  - 空间块边长，必须大于 0。

数据类型约束

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

SpaceToDepth

将输入 Tensor 的 H/W 空间块重排到 channel 维度。

公式

```
N, C, H, W = X.shape
b = blocksize
Y = np.empty((N, C * b * b, H // b, W // b), dtype=X.dtype)

for n in range(N):
    for c in range(C):
        for h in range(H):
            for w in range(W):
                out_c = c + ((h % b) * b + (w % b)) * C
                Y[n, out_c, h // b, w // b] = X[n, c, h, w]
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::pixel_unshuffle`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer
- height 和 width 必须能被 blocksize 整除。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - 量化: per-layer
  - out_channel = channel * blocksize * blocksize。
    out_height = height / blocksize。
    out_width = width / blocksize。

属性

- **blocksize - int**:
  - 空间块边长，必须大于 0。

数据类型约束

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

Resize

按指定 scales 或 sizes 对输入 Tensor 做空间尺寸缩放，支持 nearest 和 linear 两种插值模式。

公式

```
Y = Resize(X, scales, mode, coordinate_transformation_mode, nearest_mode)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::upsample_bilinear2d` , `aten::upsample_nearest2d`

输入

- **X** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **roi** - `T3` (可选):
  - Shape: N[8]

    - 裁剪区域坐标 [start_x, start_y, start_z, start_t, end_x, end_y, end_z, end_t]，仅支持默认值 [0,0,0,0,1,1,1,1]。
- **scales** - `T3` (可选):
    - Shape: N[4]
    - 各维度缩放因子 [batch_scale, channel_scale, height_scale, width_scale]，与 sizes 互斥。linear、nearest 模式下仅支持整数倍缩放，且 batch/channel 维度 scale 必须为 1，H/W 维度 scale <= 8。
- **sizes** - `T4` (可选):
    - Shape: N[4]
    - 目标输出尺寸 [batch, channel, height, width]，与 scales 互斥。linear、nearest 模式下仅支持整数倍缩放，且 batch/channel 维度不变，H/W 维度倍数 <= 8。

输出

- **Y** - `T1`:
    - Shape: NCHW[batch, channel, out_height, out_width]
    - 量化: per-layer

属性

- **mode - string**:
    - linear、nearest 模式内部通过卷积模拟实现。
- **coordinate_transformation_mode - string**:
    - 不支持 tf_crop_and_resize。
- **cubic_coeff_a - float**:
    - cubic 模式系数，当前仅支持默认值 -0.75。
- **exclude_outside - int**:
    - 不支持排除外部点。
- **extrapolation_value - float**:
    - 不支持外推值设置。
- **nearest_mode - string**:
    - nearest 模式下的取整方式。
- **antialias - int**:
    - 不支持抗锯齿。
- **keep_aspect_ratio_policy - string**:
    - 仅支持默认拉伸模式。
- **axes - int[axis_0, axis_1, axis_2, axis_3]**:
    - 指定缩放维度，当前实现未对该属性做额外约束。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T3** in (`tensor<float32>`): **T4** in (`tensor<int64>`):

---

exWindow

exWindow来源于 https://github.com/microsoft/Swin-Transformer/blob/main/models/swin_transformer.py 的window_partition和window_reverse操作。原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size × window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。

公式

```
B, C, H, W = X.shape

if mode == "partition":
    x = X.reshape(B, C, num_windows[0], window_sizes[0],
                  num_windows[1], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])

elif mode == "reverse":
    x = X.reshape(B, C, num_windows[0], num_windows[1],
                  window_sizes[0], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * window_sizes[0],
                  num_windows[1] * window_sizes[1])

elif mode == "partition_num_first":
    x = X.reshape(B, C, window_sizes[0], num_windows[0],
                  window_sizes[1], num_windows[1])
    y = np.transpose(x, (0, 1, 3, 5, 2, 4))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])
```

算子层级

RKNN3扩展算子

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - partition 要求 height = num_h * win_h，width = num_w * win_w。
    reverse 要求 height = num_h * num_w，width = win_h * win_w。

partition_num_first 要求 height = win_h * num_h，width = win_w * num_w。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, out_height, out_width]
  - 量化: per-layer
  - partition 和 partition_num_first 输出 out_height = num_h * num_w，out_width = win_h * win_w。
    reverse 输出 out_height = num_h * win_h，out_width = num_w * win_w。

属性

- **mode - string**:
  - partition 参考 window_partition 实现。
    reverse 参考 window_reverse 实现。
    partition_num_first 与 partition 的区别在于，partition_num_first 的 H/W 按照 num_windows 进行划分，而 partition 的 H/W 按照 window_sizes 进行划分。
- **num_windows - int[num_h, num_w]**:
  - num_h 范围: [1, 8192]
  - num_w 范围: [1, 8192]
  - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。
- **window_sizes - int[win_h, win_w]**:
  - win_h 范围: [1, 8192]
  - win_w 范围: [1, 8192]
  - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是基于这个大小。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

索引与散布

Gather

沿指定轴从输入张量中根据索引收集元素。

公式

```
output = Gather(input, indices, axis)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::embedding`, `aten::gather`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **indices** - `T2`:
  - Shape: INDICES[batch, index]

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **axis - int**:
  - 沿此轴收集元素。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`tensor<int32>`):

GatherElements

沿指定轴按索引从输入张量中收集元素，输出 shape 与 indices 相同。

公式

```
output = GatherElements(data, indices, axis)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::gather`

输入

- **data** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **indices** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 支持常量和非常量索引。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **axis - int**:
  - 沿此轴收集元素，负值会归一化到有效范围。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<int32>`): **T2** in (`tensor<int32>`):

GatherND

根据 indices 从输入张量中收集切片，组合成新的输出张量。

公式

```
output = GatherND(data, indices, batch_dims)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::gather_nd`

输入

- **data** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- **indices** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **batch_dims - int**:
  - 指定从开头的多少个维度作为 batch 维度，这些维度上不做 gather 操作。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ScatterND

按 indices 将 updates 散布到 data 张量的指定位置，生成输出张量，支持 reduction 归约模式。

公式

```
output = ScatterND(data, indices, updates, reduction)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::scatter_nd`

输入

- **data** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **indices** - `T2`:
  - Shape: INDICES[num_index, index_dim]
- **updates** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **reduction - string**:
  - 指定散布时的归约方式。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`):

归一化

BatchNormalization

对输入张量进行批量归一化计算（参考 https://arxiv.org/abs/1502.03167）。

公式

```
output = (input - mean) / sqrt(variance + epsilon)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::batch_norm`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **scale** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel
- **B** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel

- **mean** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel
  - 推理时与 var 一起被折叠进 scale/B，运行期不单独参与计算。
- **var** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel
  - 推理时与 mean 一起被折叠进 scale/B，运行期不单独参与计算。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **epsilon - float**:
  - 除以标准差时加上防止除 0 的小实数，默认 1e-5。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

BatchNormRelu

BatchNormalization 与 Relu 融合计算。

公式

```
output = Relu(BatchNormalization(input))
```

继承关系

继承自 BatchNormalization。

算子层级

融合算子

PYTORCH 来源

```
aten::batch_norm
```

---

exNorm

执行扩展归一化计算，并可选应用 gamma 缩放和 beta 偏移。

公式

```
x = X
weight = W if W is not None else 1
bias = B if B is not None else 0

if mode == "layer":
    x_centered = x - np.mean(x, axis=axes, keepdims=True)
    y = x_centered * fast_inv_sqrt(
        np.mean(x_centered * x_centered, axis=axes, keepdims=True) + eps)

elif mode == "mean_var":
    mean = np.mean(x, axis=axes, keepdims=True)
    var = np.mean(x * x, axis=axes, keepdims=True) - mean * mean
    y = (x - mean) * fast_inv_sqrt(var + eps)

elif mode == "rms":
    y = x * fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "pixel":
    y = x * fast_inv_sqrt(
        np.sum(x * x, axis=axes, keepdims=True) + eps)

elif mode == "l1":
    y = x / np.maximum(
        np.sum(np.abs(x), axis=axes, keepdims=True), eps)

elif mode == "l2":
    y = x / np.maximum(
        fast_sqrt(np.sum(x * x, axis=axes, keepdims=True)), eps)

elif mode == "rms_reciprocal":
    y = fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "gaussian_topk":
    mean = np.mean(x, axis=axes, keepdims=True)
    std = fast_sqrt(np.mean(x * x, axis=axes, keepdims=True) - mean * mean)
    cutoff_x = mean + std * std_multipler
    y = np.clip(x - cutoff_x, a_min=0, a_max=None).astype(x.dtype)

Y = y * weight + bias
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::layer_norm`, `aten::rms_norm`

输入

- **X** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **W** - `T1` (可选):
  - Shape: NCHW[batch, channel, height, width], [1]
  - 量化: per-layer
  - gamma 缩放参数，支持按 layer、channel 或与输出兼容的广播形状。
- **B** - `T1` (可选):
  - Shape: NCHW[batch, channel, height, width], [1]
  - 量化: per-layer
  - beta 偏移参数，支持按 layer、channel 或与输出兼容的广播形状。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - mode 为 rms_reciprocal 时，axes 指定的维度保持 keepdims 语义并输出为 1。

属性

- **mode - string**:
  - 可选值范围: layer, mean_var, rms, pixel, l1, l2, rms_reciprocal, gaussian_topk。
- **axes - int[axis0, axis1, axis2]**:
  - 默认值为 [1]。
    shape 推导支持负轴；NPU 子图实现主要支持 [1] 和 [1, 2, 3]。
- **eps - float**:
  - 默认值为 1e-5，用于避免除零。
- **std_multipler - float**:
  - 仅 gaussian_topk 使用，默认值为 1.0。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exSoftmax

对输入沿指定轴做 softmax 归一化，输出与输入形状相同。

公式

`exSoftmax(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::softmax`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 受限于 transpose 的规格限制。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 做 softmax 的轴，1 为 channel 方向，3 为 width 方向。仅支持 axis 1 和 3 是 RKNN 平台限制，ONNX 标准 axis 可为任意有效轴。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exSoftmaxMask

带 mask 的 Softmax 计算，先按 mask_value 将 mask 位置置为 -inf，再做 Softmax。

公式

`input(x, mask_t, mask_value) = x + (mask_t - 1) * inf, if mask_value = 0`
`x + mask_t * (-inf), if mask_value = 1`
`exSoftmaxMask(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::softmax`

输入

- **input** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 受限于 transpose 的规格限制。
- **mask** - `T1` (可选):
  - Shape: NCHW[1, channel, height, width]
  - channel 范围: [1, 8192]
  - width 范围: [1, 8192]
  - axis=1 时 width 无限制；axis=3/-1 时 width 取值 [1, 8192]。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 做 softmax 的轴，1 为 channel 方向，3 为 width 方向。仅支持 axis 1 和 3 是 RKNN 平台限制，ONNX 标准 axis 可为任意有效轴。
- **mask_value - int**:
  - 0 表示保留位置，1 表示需要 mask 的位置。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

矩阵运算

exMatMul

矩阵乘法操作

公式

`Y = A · B + C, C 为 bias（bias 可选）`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::matmul`, `aten::bmm`, `aten::mm`

输入

- **A** - `T1` (必选):
  - Shape: NK1N[batch, k, 1, n]
  - 量化: per-layer、per-channel、group32/64/128
- **B** - `T1` (必选):
  - Shape: NK1M[batch, k, 1, m]
  - 量化: per-layer
- **C** - `T1, T2` (可选):
  - Shape: NM1M[batch, n, 1, m]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **Y** - `T1`:
  - Shape: NM1M[batch, n, 1, m]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

---

exMatMulActivation

exMatMul 与通用 Activation 融合计算，激活类型由 Activation 子算子的类型决定。

公式

`output = Activation(exMatMul(input, weight, bias))`

继承关系

继承自 exMatMul。

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::matmul`, `aten::bmm`, `aten::mm`

LLM 扩展算子

exAttention

支持 FullAttention/SlidingAttention 两种模式的通用注意力子图算子。

公式

```
output = Matmul(Softmax(Matmul(rope(query, cos_pos_emb, sin_pos_emb), rope(key, cos_pos_emb, sin_pos_emb)) + attention_mask), value)
output = Matmul(Softmax(Matmul(query, key) + attention_mask), value)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::multi_head_attention_forward`

输入

- **query** - `T1` (必选):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
- **key** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **attention_mask** - `T1` (必选):
  - Shape: NCHW[1, seq_len, 1, seq_len]
- **cos_pos_emb** - `T1` (可选):
  - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **sin_pos_emb** - `T1` (可选):

    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **Th** - `T2` (必选):
    - Shape: N[1]
    - history token len，历史已经推理过的 token 数。
- **Tc** - `T2` (必选):
    - Shape: N[1]
    - current token len，本次推理桶内的有效的 token 数。
- **Ts** - `T2` (必选):
    - Shape: N[1]
    - system prompt token len, 对齐后的系统提示词长度，[182X, 3572] 均需要对齐到 32 的整数倍。
- **Tsr** - `T2` (必选):
    - Shape: N[1]
    - real system prompt token len, 实际的系统提示词长度，小于等于 Ts。

输出

- **output** - `T1` :
    - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]

属性

- **attention_type - string**:
    - 分别表示全局注意力和滑动窗口注意力两种模式，可选值为[FullAttention, SlidingAttention]，默认 FullAttention。
- **rope - int**:
    - 是否使用 RoPE 位置编码，0 表示不使用，1 表示使用，默认 0。
- **kvcache_store_method - string**:
    - 表示 KV cache 的存储方式，Normal 表示直接存储 float16 数据，GroupQuant 表示按组量化存储，可选值为[Normal, GroupQuant]，默认 Normal。
- **kvcache_dtype - string**:
    - 表示 KV cache 的数据类型，可选值为[Float16, Int4_to_F8, Int4_to_F16]，Float16 表示使用 float16 存储，Int4_to_F8 表示使用 4-bit 整数量化存储并在计算时转换为 8-bit 浮点数，Int4_to_F16 表示使用 4-bit 整数量化存储并在计算时转换为 16-bit 浮点数，默认 Float16。
- **kvcache_buffer_len - int**:
    - KV cache 的最大长度，单位为 token，需要小于模型的 max_position_embeddings。
- **kvcache_group_size - int**:
    - 当 kvcache_store_method 为 GroupQuant 时有效，表示分组量化的组大小；当 kvcache_store_method 为 Normal 时，默认设置为 32。

- **kvcache_residual_depth - int**:
  - 当 kvcache_store_method 为 GroupQuant 时有效，表示存满该数值 token 后才触发分组量化；当 kvcache_store_method 为 Normal 时，默认设置为 32。
- **sliding_window_size - int**:
  - 当 attention_type 为 SlidingAttention 时有效，表示滑动窗口的大小；当 attention_type 为 FullAttention 时，默认设置为 -1。

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

---

exPagedAttention

使用分页 KV cache 的注意力子图算子，支持 FullAttention、SlidingAttention 和多请求 Decode。

公式

```
Q = rope(query, cos_pos_emb, sin_pos_emb) if rope == 1 else query
K_current = rope(key, cos_pos_emb, sin_pos_emb) if rope == 1 else key
K_all, V_all = concat(历史有效 KV cache, 本轮有效 K_current/value)
logits[h,q,k] = scale * dot(Q[h,q], K_all[kv_head(h),k]) + mask[q,k]
P[h,q,k] = exp(logits[h,q,k]) / (sum_j exp(logits[h,q,j]) + (exp(sinks[h]) if
has_sink == 1 else 0))
output[h,q] = sum_k P[h,q,k] * V_all[kv_head(h),k]
```

算子层级

RKNN3扩展算子

输入

- **query** - `T1` (必选):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
  - num_attention_heads 必须是 num_key_value_heads 的整数倍，支持 MHA、MQA 和 GQA。
- **key** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, value_head_dim, 1, seq_len]
- **attention_mask** - `T1` (必选):
  - Shape: NCHW[1, seq_len, 1, seq_len]
  - 本轮 token 之间的加性掩码；历史 cache 的有效区间和 SlidingAttention 的窗口掩码由内部构图处理。
- **sinks** - `T3` (可选):

    - Shape: N[num_attention_heads]
    - has_sink=1 时紧跟 attention_mask，必须为 FP32 initializer，并按当前导入器要求在 graph input 中声明；每个 query head 一个 logit，仅加入 Softmax 分母，不参与 scale、RoPE 或 mask，不写入 KV cache，后续输入顺延。
- **cos_pos_emb** - `T1` (可选):
    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
    - rope=1 时必须提供，位于 attention_mask 或可选 sinks 之后。
- **sin_pos_emb** - `T1` (可选):
    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
    - rope=1 时必须提供，紧跟 cos_pos_emb。
- **control_variables** - `T2` (必选):
    - Shape: NC[request_num, 9]
    - 每个请求一行，必须保持九列顺序为 [Th, Tc, Ts, Tsr, longest_surviving_id, store_begin_id, Tr, Ti, request_masks]。
      Th 为本轮开始前的历史 token 数；Tc 为本轮有效 token 数，不超过 bucket_size，Decode 时为 1。
      Ts 为对齐后的系统提示词长度，需对齐到 32 的整数倍且小于 kvcache_buffer_len；Tsr 为实际系统提示词长度，满足 0 <= Tsr <= Ts。
      longest_surviving_id 为主 cache 中最早仍有效的 token 存储位置，空 cache 为 -1；
      store_begin_id 为主 cache 下一次写入的位置。
      Tr 为 residual cache 中的有效 token 数；Ti 为上轮结束的逻辑 token 位置，用于与 Th 比较并处理回退。
      request_masks 在多请求时控制本请求的 Attention 和 KVStore，0 表示跳过，非 0 表示执行；单请求时按 1 处理。
      longest_surviving_id、store_begin_id、Tr、Ti 是可回写状态，后续推理必须与对应 KV cache 一起延续；Th/Tc/Ts/Tsr 由调用方提供。
      新请求可使用 [0, Tc, Ts, Tsr, -1, 0, 0, 0, 1] 初始化，Th=0 时内部重置 cache 状态。

输出

- **output** - `T1` :
    - Shape: NCHW[num_attention_heads, value_head_dim, 1, seq_len]

属性

- **attention_type - string**:
    - 显式指定全局注意力或滑动窗口注意力模式。
- **scale - float**:
    - QK logits 的缩放系数，默认 1.0；需要标准缩放点积注意力时由调用方设置为 1/sqrt(head_dim)。
- **has_sink - int**:
    - 默认 0；1 表示存在 sinks 常量输入，FullAttention 和 SlidingAttention 均可使用。

- **rope - int**:
  - 默认 0；1 表示在内部对 query/key 应用 RoPE，需要同时提供 cos_pos_emb 和 sin_pos_emb；外置 RoPE 时设置为 0。
- **kvcache_store_method - string**:
  - 表示 KV cache 的数据类型，Float16 表示使用 float16 存储，Int4_to_F8 表示使用 4-bit 整数量化存储并在计算时转换为 8-bit 浮点数，Int4_to_F16 表示使用 4-bit 整数量化存储并在计算时转换为 16-bit 浮点数，默认 Float16。
- **kvcache_dtype - string**:
  - Normal 使用 Float16；其余类型用于 GroupQuant，后缀 F8/F16 表示计算路径使用的精度；未指定时使用编译配置。
- **kvcache_buffer_len - int**:
  - 每个请求主 KV cache 的最大 token 容量，必须为正数；未指定或设置为 -1 时使用编译配置。
- **tokens_per_page - int**:
  - 每页 KV cache 的 token 数，默认 512，必须为正数；大于 kvcache_buffer_len 时截断为该长度。FullAttention 按此大小分页；SlidingAttention 使用覆盖整个 kvcache_buffer_len 的单页。
- **kvcache_group_size - int**:
  - 当 kvcache_store_method 为 GroupQuant 时有效，表示分组量化的组大小；当 kvcache_store_method 为 Normal 时，默认设置为 32。
- **kvcache_residual_depth - int**:
  - 当 kvcache_store_method 为 GroupQuant 时有效，表示存满该数值 token 后才触发分组量化；当 kvcache_store_method 为 Normal 时，默认设置为 32。
- **request_num - int**:
  - 默认 1；实际请求数由 control_variables 的第 0 维静态确定，属性应与该维一致。
- **sliding_window_size - int**:
  - SlidingAttention 的窗口大小；FullAttention 时不生效，默认 -1。

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`): **T3** in (`tensor<float>`):

---

exSDPAttention

执行 scaled dot-product attention，可选叠加 mask。

公式

```
q = Query
k = Key
v = Value

score = MatMul(k, q) * scale
if Mask is not None:
    score = score + Mask

prob = Softmax(score, axis=1)
Y = MatMul(v, prob)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::scaled_dot_product_attention`

输入

- **Query** - `T1` (必选):
  - Shape: NCHW[batch, query_length, head, query_channel]
  - 量化: per-layer
  - q 矩阵输入，对应代码中的 query。
- **Key** - `T1` (必选):
  - Shape: NCHW[batch, key_length, head, key_channel]
  - 量化: per-layer
  - k 矩阵输入，对应代码中的 key，参与 qk_matmul。
- **Value** - `T1` (必选):
  - Shape: NCHW[batch, value_length, head, value_channel]
  - 量化: per-layer
  - v 矩阵输入。
- **Mask** - `T1` (可选):
  - Shape: NCHW[batch, key_channel, head, query_channel]
  - 量化: per-layer
  - attention mask，存在时在 softmax 前与 qk_matmul 结果相加，形状需要可广播到 qk_matmul 输出。

输出

- **SDPAOutput** - `T1`:

- Shape: NCHW[batch, query_length, head, value_channel]
- 量化: per-layer
- scaled dot-product attention 的最终输出。

属性

- **scale - float**:
  - 默认值为 1.0，作用在 qk_matmul 结果上。
- **QMatmul - string**:
  - qk_matmul 输出量化参数描述。
- **QAdd - string**:
  - mask add 输出量化参数描述，仅 Mask 存在时使用。
- **QSoftmax - string**:
  - softmax 输出量化参数描述。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exLinearAttention

Chunk Gated Delta Rule 线性注意力子图算子，按 chunk 分块计算 query/key/value 的递归线性注意力。

公式

```
output = LinearAttention(query, key, value, g, beta, initial_state)
```

算子层级

RKNN3扩展算子

输入

- **query** - `T1` (必选):
  - Shape: HD1S[H, D, 1, S]
  - 量化: per-layer
  - H 为 num_heads，D 为 head_dim，S 为 seq_len。
- **key** - `T1` (必选):
  - Shape: HD1S[H, D, 1, S]
  - 量化: per-layer
- **value** - `T1` (必选):

    - Shape: HD1S[H, D, 1, S]
    - 量化: per-layer
- **g** - `T1` (必选):
    - Shape: 1H1S[1, H, 1, S]
    - 量化: per-layer
    - gate 门控因子。
- **beta** - `T1` (必选):
    - Shape: 1H1S[1, H, 1, S]
    - 量化: per-layer
- **Tc** - `T2` (必选):
    - Shape: S[seq_len]
    - 有效 seq_len 标量。当 Tc < seq_len 时，对 query/key/value 在 seq 维做左侧 zero padding。
- **initial_state** - `T1` (可选):
    - Shape: HDD[H, D, 1, D]
    - 量化: per-layer
    - 可选的初始 recurrent state。若前端提供则直接接入。

输出

- **output** - `T1`:
    - Shape: HD1S[H, D, 1, S]
    - 量化: per-layer
- **final_state** - `T1`:
    - Shape: HDD[H, D, 1, D]
    - 量化: per-layer
    - 仅当 OutputFinalState 为 true 时输出。

属性

- **ChunkSize - int**:
    - chunk 分块大小，默认 32。seq_len 必须是 ChunkSize 的整数倍（由调用方负责 padding）。
- **UseQkL2Norm - bool**:
    - 是否对 query/key 做 L2 归一化，默认 false。
- **OutputFinalState - bool**:
    - 是否输出最终 recurrent state，默认 false。

数据类型约束

T1 in (`tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int32>`):

说明

子图算子，实际实现由 createSubGraph 在 IR 层展开，emit 仅返回成功。
batch 维固定为 1。seq_len 需在进入子图前 pad 到 ChunkSize 整数倍。

---

exRoPE

支持 RoPE 位置编码的通用子图算子。

公式

```
output = input * cos_pos_emb + rotate_half(input) * sin_pos_emb
```

算子层级

RKNN3扩展算子

输入

- **input** - `T1` (必选):
  - Shape: NCHW[batch_size, channel, 1, seq_len]
- **cos_pos_emb** - `T1` (必选):
  - Shape: NCHW[batch_size, channel, 1, position_embeddings]
  - position_embeddings == 1 or
    seq_len <= position_embeddings <= max_position_embeddings。
- **sin_pos_emb** - `T1` (必选):
  - Shape: NCHW[batch_size, channel, 1, position_embeddings]
  - position_embeddings == 1 or
    seq_len <= position_embeddings <= max_position_embeddings。
- **Th** - `T2` (可选):
  - Shape: N[1]
  - history token len，历史已经推理过的 token 数。
- **Tc** - `T2` (可选):
  - Shape: N[1]
  - current token len，本次推理桶内的有效的 token 数。
- **Ts** - `T2` (可选):
  - Shape: N[1]
  - system prompt token len, 对齐后的系统提示词长度，[182X, 3572] 均需要对齐到 32 的整数倍。

- **Tsr** - `T2` (可选):
  - Shape: N[1]
  - real system prompt token len, 实际的系统提示词长度，小于等于 Ts。

输出

- **output** - `T1`:
  - Shape: NCHW[batch_size, channel, 1, seq_len]

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

说明

当 exRoPE 用于 llm 模型，并传入全量的 cos/sin_pos_emb 时(position embeddings > seq_len)，会存在 Th/Tc/Ts/Tsr 等输入，exRoPE 内部会根据 Th/Tc/Ts/Tsr 计算出实际需要使用的 cos/sin_pos_emb 的长度，从而支持动态序列长度的推理。其他情况 seq_len == position_embeddings，Th/Tc/Ts/Tsr 不存在。

---

exSum

多输入加权求和，常用于 LoRA 多分支结果合并。

公式

`Y = Sum(X_i * scale_i) for i in 0..N-1`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::sum`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 可变长度输入列表，至少 1 个，所有输入形状必须一致。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer

属性

- **scales - float[scale_1, ..., scale_N-1]**:
  - 每个额外输入（除第一个外）的加权系数，数量为输入个数 - 1。

数据类型约束

T1 in (`tensor<float16>`):

类型转换

Cast

将输入张量的元素逐个转换为指定目标数据类型。

公式

`output = Cast(input, to)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::to`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

输出

- **output** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

属性

- **to - int**:
  - 目标数据类型，由 ONNX to_type 映射而来，支持 bool/int8/uint8/int16/int32/float16/float32。

数据类型约束

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

---

exDataConvert

将输入张量的数据类型转换为指定目标类型。

公式

```
output = exDataConvert(input, input_type, output_type)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::to`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

输出

- **output** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

属性

- **input_type - int**:
  - 输入数据类型。
- **output_type - int**:
  - 输出数据类型。

数据类型约束

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

输入输出处理

exInputProc

执行输入数据的预处理。

算子层级

RKNN3扩展算子

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

输出

- **B** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`): **T2** in (`tensor<float16>` , `tensor<int8>`):

---

exOutputProc

执行输出数据的后处理。

算子层级

RKNN3扩展算子

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

输出

- **B** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<float16>` , `tensor<int8>`): **T2** in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`):

RK3572

卷积

Conv

卷积计算。

公式

`output = Conv(input_tensor, weight, bias)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::_convolution`

输入

- **input_tensor** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **weight** - `T1` (必选):
  - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
  - 量化: per-layer, per-channel
  - kernel_h 范围: [1, 31]
  - kernel_w 范围: [1, 31]
- **bias** - `T1` (可选):
  - Shape: C[output_channel]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **strides - int[stride_h, stride_w]**:

    - stride_h 范围: [1, 7]
    - stride_w 范围: [1, 7]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
    - pads_top 范围: [0, 15]
    - pads_left 范围: [0, 15]
    - pads_bottom 范围: [0, 15]
    - pads_right 范围: [0, 15]
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h 范围: [1, 31]
    - dilation_w 范围: [1, 31]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

特殊情况

Depthwise Convolution 当 Conv 按 depthwise 卷积分组语义使用时，适用以下差异规格。

- **输入差异** — **weight**：kernel_h 范围: [1, 8]；kernel_w 范围: [1, 8]
- **属性差异** — **dilations**：dilation_h 范围: [1, 32]；dilation_w 范围: [1, 32]

ConvTranspose

转置卷积计算。

公式

```
output = ConvTranspose(input_tensor, weight, bias)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::conv_transpose1d` , `aten::conv_transpose2d`

输入

- **input_tensor** - `T1` (必选):
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

- 当 dilated_kernel_h > 1 时，input_tensor.width < 16383。
  首层输入 width 还需满足模型输入限制。

- **weight** - `T1` (必选):
  - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
  - 量化: per-layer, per-channel
  - kernel_h 范围: [1, 31]
  - kernel_w 范围: [1, 31]
- **bias** - `T1` (可选):
  - Shape: C[output_channel]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **strides - int[stride_h, stride_w]**:
  - stride_h 范围: [1, 8]
  - stride_w 范围: [1, 8]
- **pads - int[pads_top, pads_left, pads_bottom, pads_right]**:
  - pads_top 范围: [0, 15]
  - pads_left 范围: [0, 15]
  - pads_bottom 范围: [0, 15]
  - pads_right 范围: [0, 15]
  - 不支持 kernel_h * dilation_h - dilation_h - pads_top < 0
    不支持 kernel_w * dilation_w - dilation_w - pads_left < 0
    不支持 stride_h * (height - 1) - pads_top + 1 < output_h
    不支持 stride_w * (width - 1) - pads_left + 1 < output_w
- **group - int**:
  - 支持 group = 1；当 num_input = num_output 时，也支持 group = num_output。
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h 范围: [1, 32]
  - dilation_w 范围: [1, 32]
  - 当 dilation != (1, 1) 时，仅支持 kernel = (1, 1)。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

ConvActivation

Conv 与通用 Activation 融合计算，激活类型由 Activation 子算子的类型决定。

公式

```
output = Activation(Conv(input_tensor, weight, bias))
```

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvRelu

Conv 与 Relu 融合计算。

公式

```
output = Relu(Conv(input_tensor, weight, bias))
```

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvClip

Conv 与 Clip 融合计算。

公式

`output = Clip(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvSigmoid

Conv 与 Sigmoid 融合计算。

公式

`output = Sigmoid(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvTanh

Conv 与 Tanh 融合计算。

公式

`output = Tanh(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvElu

Conv 与 Elu 融合计算。

公式

`output = Elu(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvHardSigmoid

Conv 与 HardSigmoid 融合计算。

公式

`output = HardSigmoid(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvHardSwish

Conv 与 HardSwish 融合计算。

公式

`output = HardSwish(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvSwish

Conv 与 Swish 融合计算。

公式

`output = Swish(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvSoftplus

Conv 与 Softplus 融合计算。

公式

`output = Softplus(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvMish

Conv 与 Mish 融合计算。

公式

`output = Mish(Conv(input_tensor, weight, bias))`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvAdd

Conv 与 Add 融合计算。

公式

`output = Add(Conv(input_tensor, weight, bias), residual)`

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

额外输入

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- Add 的第二输入。
  支持广播，广播约束同 Add。

ConvAddRelu

Conv 与 Add 及 Relu 融合计算。

公式

```
output = Relu(Add(Conv(input_tensor, weight, bias), residual))
```

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

```
aten::_convolution
```

额外输入

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - Add 的第二输入。
    支持广播，广播约束同 Add。

ConvPRelu

Conv 与 PRelu 融合计算。

公式

```
output = PRelu(Conv(input_tensor, weight, bias))
```

继承关系

继承自 Conv。

算子层级

融合算子

PYTORCH 来源

`aten::_convolution`

ConvTransposeActivation

ConvTranspose 与通用 Activation 融合计算，激活类型由 Activation 子算子的类型决定。

公式

```
output = Activation(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeRelu

ConvTranspose 与 Relu 融合计算。

公式

```
output = Relu(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeClip

ConvTranspose 与 Clip 融合计算。

公式

`output = Clip(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeSigmoid

ConvTranspose 与 Sigmoid 融合计算。

公式

`output = Sigmoid(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeTanh

ConvTranspose 与 Tanh 融合计算。

公式

`output = Tanh(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeElu

ConvTranspose 与 Elu 融合计算。

公式

```
output = Elu(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeHardSigmoid

ConvTranspose 与 HardSigmoid 融合计算。

公式

```
output = HardSigmoid(ConvTranspose(input_tensor, weight, bias))
```

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeHardSwish

ConvTranspose 与 HardSwish 融合计算。

公式

`output = HardSwish(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeSwish

ConvTranspose 与 Swish 融合计算。

公式

`output = Swish(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeSoftplus

ConvTranspose 与 Softplus 融合计算。

公式

`output = Softplus(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeMish

ConvTranspose 与 Mish 融合计算。

公式

`output = Mish(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

ConvTransposeAdd

ConvTranspose 与 Add 融合计算。

公式

`output = Add(ConvTranspose(input_tensor, weight, bias), residual)`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

额外输入

- **residual** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- Add 的第二输入。
  支持广播，广播约束同 Add。

ConvTransposePRelu

ConvTranspose 与 PRelu 融合计算。

公式

`output = PRelu(ConvTranspose(input_tensor, weight, bias))`

继承关系

继承自 ConvTranspose。

算子层级

融合算子

PYTORCH 来源

`aten::conv_transpose2d`

---

exConvStreaming

带流式 cache 的卷积计算。

公式

`y = Conv(Concat(cache, x), w, b)`
`cache = Slice(Concat(cache, Slice(x, width=[0, tc])), width=[end - cache_w, end])`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::_convolution`

输入

- **x** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **w** - `T1` (必选):

    - Shape: OIHW[output_channel, input_channel, kernel_h, kernel_w]
    - 量化: per-layer, per-channel
    - kernel_h 范围: [1, 31]
    - kernel_w 范围: [1, 31]
- **b** - `T1` (可选):
    - Shape: C[output_channel]
    - 量化: per-layer
    - 可缺省；无 bias 且需要传 tc 时，b 可为空输入占位。
- **tc** - `T2` (可选):
    - Shape: N[1]
    - tc 表示本次写入 cache 的有效宽度，长度需小于输入 width。
      输入个数为 3 时，第三个输入若尺寸等于 weight 的 output_channel，则按 b 处理，否则按 tc 处理。

输出

- **y** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

属性

- **strides - int[stride_h, stride_w]**:
    - stride_h 范围: [1, 7]
    - stride_w 范围: [1, 7]
- **pads - int[pads_top, cache_w, pads_bottom, pads_right]**:
    - pads_top 范围: [0, 15]
    - pads_bottom 范围: [0, 15]
    - pads_right 范围: [0, 15]
    - cache_w 为流式 cache 长度，必须大于 0。
- **group - int**: —
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h 范围: [1, 31]
    - dilation_w 范围: [1, 31]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

特殊情况

Depthwise exConvStreaming 当 exConvStreaming 按 depthwise 卷积分组语义使用时，适用以下差异规格。

- **输入差异** — **w**：kernel_h 范围: [1, 8]；kernel_w 范围: [1, 8]
- **属性差异** — **dilations**：dilation_h 范围: [1, 32]；dilation_w 范围: [1, 32]

说明

子图算子，实际实现由 createSubGraph 在 IR 层展开，emit 仅返回成功。

池化

AveragePool

按指定 kernel_shape、strides 和 pads 对输入 Tensor 做平均池化。

公式

```
Y = average_pool(X, kernel_shape, strides, pads)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::adaptive_avg_pool2d`, `aten::avg_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8/int16 量化需满足 height <= 32 且 width <= 32

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **kernel_shape - int[kernel_h, kernel_w]**:

    - kernel_h 范围: [1, 32]
    - kernel_w 范围: [1, 32]
- **strides - int[stride_h, stride_w]**:
    - stride_h 范围: [1, 63]
    - stride_w 范围: [1, 63]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
    - pad_top 范围: [0, 63]
    - pad_left 范围: [0, 63]
    - pad_bottom 范围: [0, 63]
    - pad_right 范围: [0, 63]
- **auto_pad - string**:
    - 仅支持显式 pads。
- **ceil_mode - int**:
    - ceil 模式通过补充 bottom/right padding 支持。
- **count_include_pad - int**:
    - 为 1 时均值计算包含 padding 区域。
- **dilations - int[dilation_h, dilation_w]**:
    - dilation_h 范围: [1, 1]
    - dilation_w 范围: [1, 1]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

MaxPool

按指定 kernel_shape、strides 和 pads 对输入 Tensor 做最大池化。

公式

`Y = max_pool(X, kernel_shape, strides, pads)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::max_pool1d` , `aten::max_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **kernel_shape - int[kernel_h, kernel_w]**:
  - kernel_h 范围: [1, 32]
  - kernel_w 范围: [1, 256]
  - kernel_h * kernel_w <= 256
- **strides - int[stride_h, stride_w]**:
  - stride_h 范围: [1, 31]
  - stride_w 范围: [1, 256]
- **pads - int[pad_top, pad_left, pad_bottom, pad_right]**:
  - pad_top 范围: [0, 63]
  - pad_left 范围: [0, 63]
  - pad_bottom 范围: [0, 63]
  - pad_right 范围: [0, 63]
- **auto_pad - string**:
  - 仅支持显式 pads。
- **ceil_mode - int**:
  - ceil 模式通过补充 bottom/right padding 支持。
- **dilations - int[dilation_h, dilation_w]**:
  - dilation_h 范围: [1, 1]
  - dilation_w 范围: [1, 1]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

GlobalAveragePool

对输入 Tensor 的 H/W 空间维度做全局平均池化。

公式

`Y = mean(X, axes=[2, 3])`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::adaptive_avg_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8 量化需满足 height <= 32 且 width <= 32；

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, 1, 1]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

GlobalMaxPool

对输入 Tensor 的空间维度做全局最大池化，输出每个通道的最大值。

公式

`Y[batch, channel, 0, 0] = max(X[batch, channel, :, :])`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::adaptive_max_pool2d`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 硬约束：height * width <= 256 且 height <= 32。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, 1, 1]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

元素级运算

Add

执行元素级二进制加法，支持 Numpy 样式广播。

公式

`C = A + B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::add` , `aten::add_`

输入

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

    - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<int32>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Sub

执行元素级二进制减法，支持 Numpy 样式广播。

公式

`C = A - B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sub` , `aten::sub_`

输入

- **A** - `T1, T2`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer
    - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer
    - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Mul

执行元素级二进制乘法，支持 Numpy 样式广播。

公式

`C = A * B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::mul` , `aten::mul_`

输入

- **A** - `T1, T2` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Div

执行元素级二进制除法，支持 Numpy 样式广播。

公式

`C = A / B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::div`, `aten::true_divide`

输入

- **A** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1, T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`const_tensor<float16>`):

Min

对多个输入张量逐元素求最小值，支持 Numpy 样式广播。

公式

`C = min(A, B, ...)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::min`, `aten::minimum`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer, per-channel

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Max

对多个输入张量逐元素求最大值，支持 Numpy 样式广播。

公式

`C = max(A, B, ...)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::max`, `aten::maximum`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

    - 量化: per-layer
    - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer
    - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer, per-channel

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Pow

执行元素级指数运算。

公式

`output = input ^ exponent`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::pow` , `aten::pow_`

输入

- **input** - `T1`:
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer
- **exponent** - `T2`:
    - Shape: S[scalar]

输出

- **output** - `T1`:

- Shape: NCHW[batch, channel, height, width]
- 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Neg

对输入张量执行元素级取负操作。

公式

`Y = -X`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::neg` , `aten::neg_`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Exp

对输入张量逐元素计算以 e 为底的指数。

公式

`output = exp(input)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::exp`, `aten::exp_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Sqrt

对输入张量执行元素级平方根运算。

公式

```
output = sqrt(input)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sqrt`, `aten::sqrt_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Reciprocal

对输入张量执行元素级倒数运算。

公式

```
Y = 1 / X
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::reciprocal` , `aten::reciprocal_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Floor

对输入张量逐元素向下取整（向负无穷方向取最接近的整数）。

公式

`output = floor(input)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::floor`, `aten::floor_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Round

取整函数，采用 round-half-up 语义，公式为 y = Floor(x + 0.5)。与 ONNX Round 的 round-half-to-even（银行家舍入）不同。

公式

`y = Floor(x + 0.5)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::round`, `aten::round_`

输入

- **x** - `T1`:

- Shape: NCHW[batch, channel, height, width]

输出

- **y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<float16>`):

Sin

按元素计算正弦函数。

公式

`y = sin(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sin` , `aten::sin_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

说明

实现先对输入做 mod 2π，再查 LUT 表。

Cos

按元素计算余弦函数。

公式

`y = cos(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::cos`, `aten::cos_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

说明

实现先对输入做 mod 2π，再查 LUT 表。

Atan

按元素计算反正切函数。

公式

`y = atan(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::atan`, `aten::atan_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

说明

LUT 主表覆盖 x ∈ [-16, 16]，该范围外按斜率 1/(1+x^2) 插值逼近，输出收敛于 ±π/2。

Tan

按元素计算正切函数。

公式

`y = tan(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::tan`, `aten::tan_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<float16>` , `tensor<bfloat16>`):

说明

实现先对输入做 mod π（结果区间 [-π/2, π/2]），再查 LUT 表。LUT 主表仅覆盖 [-1, 1]，该范围外输出钳位为 tan(±1)，因此大输入（mod π 后超出 [-1, 1]）无法保证精度。

---

exFastExp

按元素快速计算自然指数，近似实现以提升性能。

公式

`output ≈ exp(input)`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::exp` , `aten::exp_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exTwoPow

按元素计算 2 的幂次方。

公式

`y = 2^x`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::pow`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

AddRelu

Add 与 Relu 融合计算。

公式

`C = Relu(A + B)`

继承关系

继承自 Add。

算子层级

融合算子

PYTORCH 来源

`aten::add`, `aten::add_`

MulRelu

Mul 与 Relu 融合计算。

公式

```
C = Relu(A * B)
```

继承关系

继承自 Mul。

算子层级

融合算子

PYTORCH 来源

`aten::mul`, `aten::mul_`

激活函数

Relu

整流线性单元，对输入逐元素做 max(input, 0)。

公式

```
output = max(input, 0)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::relu`, `aten::relu6`, `aten::relu_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Sigmoid

给定输入张量，按元素应用 Sigmoid 函数。

公式

```
y = 1 / (1 + exp(-x))
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sigmoid` , `aten::sigmoid_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Clip

将输入张量的元素裁剪到 [min, max] 区间，超出边界的值被限制到边界。

公式

`output = Clip(input, min, max)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::clamp`, `aten::clamp_`

输入

- **input** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **min** - `T2` (可选):
  - Shape: S[scalar]
  - 裁剪下界，标量。缺省表示不限制下界。
- **max** - `T2` (可选):
  - Shape: S[scalar]
  - 裁剪上界，标量。缺省表示不限制上界。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

LeakyRelu

Leaky 整流线性单元，正值保持不变，负值按固定比例 alpha 缩放。

公式

`output = input if input >= 0 else alpha * input`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::leaky_relu`, `aten::leaky_relu_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **alpha - float[alpha]**:
  - alpha 范围: [0, 1]
  - 负值缩放比例。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

PRelu

参数化整流线性单元，正值保持不变，负值按可学习参数 slope 缩放。

公式

```
output = input if input >= 0 else slope * input
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::prelu`, `aten::prelu_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

    - 量化: per-layer, per-channel
- **slope** - `T2` :
    - Shape: C[channel]
    - 量化: per-channel
    - 支持单个标量或 C 维度系数。

输出

- **output** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer, per-channel

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

Elu

按元素应用 Elu 函数，alpha = 1。

公式

`f(x) = alpha * (exp(x) - 1) for x < 0; f(x) = x for x >= 0`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::elu` , `aten::elu_`

输入

- **input** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

输出

- **output** - `T1` :
    - Shape: NCHW[batch, channel, height, width]
    - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Softplus

按元素应用 Softplus 函数。

公式

`y = ln(exp(x) + 1)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::softplus`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

HardSigmoid

按元素应用 HardSigmoid 函数，由 alpha、beta 属性控制。

公式

`y = max(0, min(1, alpha * x + beta))`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::hardsigmoid`, `aten::hardsigmoid_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **alpha - float**:
  - 默认 0.2（与 ONNX 标准一致）。
- **beta - float**:
  - 默认 0.5。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

HardSwish

按元素应用 HardSwish 函数，alpha = 1/6，beta = 0.5。

公式

```
y = x * max(0, min(1, alpha * x + beta))
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::hardswish`, `aten::hardswish_`

输入

- **X** - `T1`:

- Shape: NCHW[batch, channel, height, width]
- 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Tanh

计算给定输入张量按元素的双曲正切。

公式

`y = tanh(x)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::tanh` , `aten::tanh_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Mish

按元素应用 Mish 函数。

公式

`y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::mish`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exGelu

按元素应用 Gelu 函数。

公式

`y = x * Phi(x)，当 Phi 取 tanh 近似时 y = 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715 * x^3)))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::gelu`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exHardSwish

按元素应用 HardSwish 函数，alpha = 1/6，beta = 0.5。

公式

`y = x * max(0, min(1, alpha * x + beta))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::hardswish`, `aten::hardswish_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exMish

按元素应用 Mish 函数。

公式

```
y = x * tanh(softplus(x)) = x * tanh(ln(1 + exp(x)))
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::mish` , `aten::mish_`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exSwish

按元素应用 Swish 函数（也称 SiLU）。

公式

```
y = x * sigmoid(x)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::silu`, `aten::silu_`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

---

exGlu

将输入 Tensor 在指定 axis 上均分为两部分，并对第二部分做 sigmoid 门控后与第一部分相乘。

公式

```
a, b = np.split(X, 2, axis=axis)
Y = a * sigmoid(b)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::glu`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - axis 指定维度的长度必须为偶数。
    NPU 子图实现主要面向 axis = 1 的 channel 维度拆分。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, out_channel, height, width]
  - 量化: per-layer
  - 输出 shape 与输入 shape 保持一致，但 axis 指定维度变为输入的一半。
    当 axis = 1 时，out_channel = channel / 2。

属性

- **axis - int**:
  - 指定 GLU 拆分的维度，取值范围为 [-rank, rank - 1]。
    负轴会按 Numpy 语义转换为正轴。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exLogit

对输入张量执行 logit 激活函数计算。

公式

```
Y = log(X / (1 - X))
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::logit`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

比较与逻辑运算

Equal

逐元素比较两个张量是否相等，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = (A == B)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::eq` , `aten::eq_`

输入

- **A** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **B** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **C** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - bool 语义结果，RKNN 以 int8 内存布局存储。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Greater

按元素比较 A 是否大于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = (A > B)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::gt`, `aten::gt_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`tensor<bool>`, `tensor<int8>`):

GreaterOrEqual

逐元素比较 A 是否大于等于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = (A >= B)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::ge`, `aten::ge_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - bool 语义结果，RKNN 以 int8 内存布局存储。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Less

按元素比较 A 是否小于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

```
C = (A < B)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::lt` , `aten::lt_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- 支持 Numpy 样式广播。

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`tensor<bool>`, `tensor<int8>`):

LessOrEqual

按元素比较 A 是否小于等于 B，输出 bool 类型结果，支持 Numpy 样式广播。

公式

```
C = (A <= B)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::le`, `aten::le_`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。

输出

- **C** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<bool>` , `tensor<int8>`):

Not

对输入张量执行元素级逻辑取反操作。

公式

`Y = !X`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::logical_not`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

输出

- **Y** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<bool>`):

And

执行元素级逻辑与运算，输出 bool 类型结果，支持 Numpy 样式广播。

公式

`C = A && B`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::bitwise_and`, `aten::logical_and`

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。
- **B** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **C** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<bool>`, `tensor<int8>`):

Where

根据 condition 条件选择 x 或 y 中的元素，支持 Numpy 样式广播。

公式

```
output = select(condition, x, y)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::where`

输入

- **condition** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
- **x** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- 支持 Numpy 样式广播。

- **y** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 支持 Numpy 样式广播。

输出

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<bool>`): **T2** in (`tensor<int8>` , `tensor<float16>`):

归约运算

ReduceMax

沿指定轴对输入张量求最大值，支持多轴与 keepdims。

公式

`output = ReduceMax(input, axes, keepdims)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::amax` , `aten::max`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- keepdims=1 时被缩减的轴大小为 1，keepdims=0 时该轴从输出中消失，实际输出 rank 依 axes 和 keepdims 而定。

属性

- **axes - int[]**:
  - 计算最大值的轴。
- **keepdims - int**:
  - 是否保留缩减维度（1 保留，0 不保留）。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

ReduceMean

按指定 axes 对输入 Tensor 做均值归约。

公式

`Y = mean(data, axes)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::mean`

输入

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8/int16 量化且 axes = [2]、[3]、[2, 3]、[1, 2, 3] 时需满足 height <= 32 且 width <= 32
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - 指定需要归约的维度， 负轴会按 Numpy 语义转换为正轴。
    支持 axes = [1]、[2]、[3]、[2, 3]、[1, 2, 3]

输出

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ReduceSum

按指定 axes 对输入 Tensor 做求和归约。

公式

`Y = sum(data, axes)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::sum`

输入

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - int8/int16 量化且 axes = [2]、[3]、[2, 3]、[1, 2, 3] 时需满足 height <= 32 且 width <= 32
- **axes** - `T2` :
  - Shape: Axes[num_axes]
  - 指定需要归约的维度， 负轴会按 Numpy 语义转换为正轴。
    支持 axes = [1]、[2]、[3]、[2, 3]、[1, 2, 3]

输出

- **reduced** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`const_tensor<int64>`):

ArgMax

沿指定轴返回最大值的索引。

公式

`output = ArgMax(input, axis)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::argmax`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 指定在哪个轴上进行 argmax 操作，取值范围 [0, ndim-1]。
- **keepdims - int**:
  - 是否保持输出维度与输入相同。0 表示不保持，1 表示保持。
- **select_last_index - int**:
  - 当有多个最大值时，是否选择最后一个索引。0 表示选择第一个，1 表示选择最后一个。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ArgMin

沿指定轴返回最小值的索引。

公式

`output = ArgMin(input, axis)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::argmin`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 指定在哪个轴上进行 argmin 操作，取值范围 [0, ndim-1]。
- **keepdims - int**:
  - 是否保持输出维度与输入相同。0 表示不保持，1 表示保持。
- **select_last_index - int**:
  - 当有多个最小值时，是否选择最后一个索引。0 表示选择第一个，1 表示选择最后一个。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

TopK

沿指定轴检索输入张量中最大（或最小，由 largest 决定）的 K 个值及其索引。

公式

`(values, indices) = TopK(X, K, axis, largest, sorted)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::topk`

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **K** - `T3` :
  - Shape: S[scalar]
  - 取最大的 K 个值，标量，存储在 int64 张量中。

输出

- **values** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 最大的 K 个值。
- **indices** - `T2` :
  - Shape: NCHW[batch, channel, height, width]
  - 对应值的索引。ONNX 标准为 int64，RKNN 实际输出 int32 或 int64。

属性

- **axis - int**:
  - 计算的轴，默认 -1。
- **largest - int**:
  - 1 取最大，0 取最小，默认 1。
- **sorted - int**:
  - 1 按降序输出，0 不保证顺序，默认 1。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`): **T3** in (`tensor<int64>`):

张量变换

Reshape

在不改变输入数据和元素数量的情况下，返回一个具有指定形状的张量。

公式

`output = reshape(input, shape)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::reshape`, `aten::reshape_`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **shape - int**:
  - 指定输出张量的形状。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`):

Transpose

对输入张量按 perm 指定的轴顺序进行转置。

公式

`Y = transpose(X, perm)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::permute`, `aten::transpose`, `aten::transpose_`

输入

- **X** - `T1, T2`:
  - Shape: NCHW[n1, c1, h1, w1]
  - 量化: per-layer
  - 详细约束规则见属性列表 perm。

输出

- **Y** - `T1, T2`:
  - Shape: NCHW[n2, c2, h2, w2]
  - 量化: per-layer
  - 详细约束规则见属性列表 perm。

属性

- **perm - int[d0, d1, d2, d3]**:
  - 转置的轴顺序。不同 perm 参数的约束见下方 perm 约束表。

数据类型约束

T1 in (`tensor<int8>`): **T2** in (`tensor<int16>`, `tensor<float16>`):

说明

不同 perm 参数限制如下（其中 p = 268435456，q = 65536）：

perm	T1 约束	T2 约束
[0,1,2,3]	无限制	无限制
[0,1,3,2]	h1%16=0, h1*w1<16*p, h1*w1<2*q	h1%8=0, h1*w1<8*p, h1*w1<2*q
[0,2,3,1]	c1<8192, h1*w1%8=0, w1*c1<p*16	c1<8192, h1*w1%8=0, w1*c1<p*8
[0,2,1,3]	h1%16=0, h1*w1<16*p, h1*w1<2*q, c1*w1<p	h1%8=0, h1*w1<8*p, h1*w1<2*q, c1*w1<p
[0,3,1,2]	h1*w1<p, w1<8192, h1*c1<p	h1*w1<p, w1<8192, h1*c1<p
[0,3,2,1]	h1%16=0, h1*w1<16*p, h1*w1<2*q, c1<p*16	h1%8=0, h1*w1<8*p, h1*w1<2*q, c1<p*8
[1,0,2,3]	h1*w1<16*p, n1<8192, h1*w1<p	h1*w1<8*p, n1<8192, h1*w1<p
[1,0,3,2]	h1%16=0, c1*h1*w1<16*p, h1*w1<2*q	h1%8=0, c1*h1*w1<8*p, h1*w1<2*q

perm	T1 约束	T2 约束
[1,2,0,3]	n1*w1*<*p*16, c1*h1*w1<p, w1%16=0	n1*w1*<*p*8, c1*h1*w1<p, w1%8=0
[1,2,3,0]	h1*w1*<*p, n1*<*p*16, c1*h1*w1<p*16, n1*w1<p*16	h1*w1*<*p, n1*<*p*8, c1*h1*w1<p*8, n1*w1<p*8
[1,3,0,2]	c1*w1*<*8192, n1*h1<p, w1%16=0	c1*w1*<*8192, n1*h1<p, w1%8=0
[1,3,2,0]	c1*h1*w1<p, n1*h1<p, w1<8192, w1%16=0	c1*h1*w1<p, n1*h1<p, w1<8192, w1%8=0
[2,1,0,3]	c1<8192, c1*h1*w1<p, n1*w1<p	c1<8192, c1*h1*w1<p, n1*w1<p
[2,1,3,0]	c1*h1*w1<p, n1*w1*<*p*16, c1<8192, n1*w1%4=0	c1*h1*w1<p, n1*w1*<*p*8, c1<8192, n1*w1%4=0
[2,0,3,1]	c1<8192, c1*h1*w1<p	c1<8192, c1*h1*w1<p
[2,0,1,3]	n1*c1*h1*w1*<*p, h1*w1%16=0	n1*c1*h1*w1*<*p, h1*w1%8=0
[2,3,1,0]	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%16=0	c1*h1*w1<p, n1*c1*<*p, h1*w1<8192, h1*w1%8=0
[2,3,0,1]	h1*w1*<*8192, n1*c1<p	h1*w1*<*8192, n1*c1<p
[3,0,2,1]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*w1%8=0
[3,0,1,2]	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%16=0	h1*w1*<*p, n1*c1<8192, c1*h1*<*p, n1*w1<8192, n1*h1*w1%8=0
[3,2,0,1]	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*c1*w1<p
[3,2,1,0]	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*16, h1%16=0	c1*h1*w1<p, h1*w1*<*8192, 2*h1*w1*<*q, n1*c1<p*8, h1%8=0
[3,1,0,2]	h1*w1*<*p, c1*w1<8192, n1*h1<p	h1*w1*<*p, c1*w1<8192, n1*h1<p
[3,1,2,0]	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p	h1*w1*<*p, n1*c1<8192, n1*h1*<*p, n1*c1*w1<p

---

Concat

沿指定 axis 将多个张量拼接成一个张量，所有输入除 axis 维外大小必须相同。

公式

```
output = Concat(input_tensor_list, axis)
```

算子层级

标准ONNX算子

PYTORCH 来源

```
aten::cat
```

输入

- **input_tensor** - `T`:
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer
- 输入为张量列表，各张量除 axis 维外大小必须相同。

输出

- **output** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **axis - int[axis]**:
  - axis 范围: [-4, 3]
  - 拼接轴，RKNN 限制输入为 4D NCHW，支持负轴。

数据类型约束

T in (`tensor<int8>` , `tensor<float16>`):

Split

沿指定 axis 将输入张量切分成多个输出张量。

公式

```
outputs = Split(input_tensor, axis, num_outputs, split)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::split`

输入

- **input_tensor** - `T`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T`:
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer
- 一或多个输出张量，数量由 num_outputs 决定，shape 按切分情况决定。

属性

- **axis - int[axis]**:
  - axis 范围: [-4, 3]
  - 切分轴，RKNN 限制输入为 4D NCHW，支持负轴。
- **num_outputs - int**: —
- **split - int[]**:
  - 指定切分后各输出在 axis 方向上的大小。

数据类型约束

T in (`tensor<int8>` , `tensor<float16>`):

Expand

将输入张量广播到目标形状。

公式

```
output = Expand(input, target_shape)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::expand` , `aten::expand_as`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

Slice

按 numpy 切片规则获取输入张量的一个切片。

公式

```
output = Slice(data, starts, ends, axes, steps)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::narrow` , `aten::slice`

输入

- **data** - `T` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **starts** - `T2` (必选):
  - Shape: D[dim]
  - 各轴切分起始位置。
- **ends** - `T2` (必选):
  - Shape: D[dim]
  - 各轴切分终止位置。
- **axes** - `T2` (可选):
  - Shape: D[dim]
  - 选取切分的轴，支持 0~3 轴，支持同时多轴选择；缺省时按全部轴处理。
- **steps** - `T2` (可选):
  - Shape: D[dim]
  - 各轴切分步长。当 height*width==1 且 channel%subc==0 且 starts[1]%subc==0 且 ends[1]%subc==0 且 steps[1]<=subc 时，steps 可不为 1； 否则仅当 steps==[1,1,1,1] 时可通过 NPU 运行。

输出

- **output** - `T` :

- Shape: NCHW[batch, channel, height, width]
- 量化: per-layer

数据类型约束

T in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int64>`):

Pad

按指定 pads 和模式对输入张量进行填充。

公式

```
output = Pad(data, pads, mode, constant_value)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::constant_pad_nd`

输入

- **data** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **pads** - `T2` (必选):
  - Shape: PADS[batch_begin, channel_begin, height_begin, width_begin, batch_end, channel_end, height_end, width_end]
- **constant_value** - `T1` (可选):
  - Shape: S[scalar]
  - 量化: per-layer
  - 与 data 同类型，仅 constant 模式生效，缺省为 0。

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **mode - string**:
  - constant 模式无范围限制； edge 模式仅支持 h/w 方向。

数据类型约束

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int64>`):

特殊情况

Reflect Pad reflect 模式下输入 shape 分量有额外范围限制。

- **输入差异 — data**：channel 范围: [1, 8192]；height 范围: [1, 8192]；width 范围: [1, 8176]

DepthToSpace

将输入 Tensor 的 channel 维度按 blocksize 重排回 H/W 空间维度。

公式

```
N, C, H, W = X.shape
b = blocksize
out_channel = C // (b * b)
Y = np.empty((N, out_channel, H * b, W * b), dtype=X.dtype)

for n in range(N):
    for c in range(out_channel):
        for h in range(H * b):
            for w in range(W * b):
                offset = (h % b) * b + (w % b)
                in_c = c + offset * out_channel
                Y[n, c, h, w] = X[n, in_c, h // b, w // b]
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::pixel_shuffle`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - channel 必须能被 blocksize * blocksize 整除。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - 量化: per-layer
  - out_channel = channel / (blocksize * blocksize)。
    out_height = height * blocksize。
    out_width = width * blocksize。

属性

- **blocksize - int**:
  - 空间块边长，必须大于 0。

数据类型约束

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

SpaceToDepth

将输入 Tensor 的 H/W 空间块重排到 channel 维度。

公式

```
N, C, H, W = X.shape
b = blocksize
Y = np.empty((N, C * b * b, H // b, W // b), dtype=X.dtype)

for n in range(N):
    for c in range(C):
        for h in range(H):
            for w in range(W):
                out_c = c + ((h % b) * b + (w % b)) * C
                Y[n, out_c, h // b, w // b] = X[n, c, h, w]
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::pixel_unshuffle`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer
- height 和 width 必须能被 blocksize 整除。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, out_channel, out_height, out_width]
  - 量化: per-layer
  - out_channel = channel * blocksize * blocksize。
    out_height = height / blocksize。
    out_width = width / blocksize。

属性

- **blocksize - int**:
  - 空间块边长，必须大于 0。

数据类型约束

T1 in (`tensor<int8>` , `tensor<int16>` , `tensor<float16>`):

Resize

按指定 scales 或 sizes 对输入 Tensor 做空间尺寸缩放，支持 nearest 和 linear 两种插值模式。

公式

```
Y = Resize(X, scales, mode, coordinate_transformation_mode, nearest_mode)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::upsample_bilinear2d` , `aten::upsample_nearest2d`

输入

- **X** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **roi** - `T3` (可选):
  - Shape: N[8]

    - 裁剪区域坐标 [start_x, start_y, start_z, start_t, end_x, end_y, end_z, end_t]，仅支持默认值 [0,0,0,0,1,1,1,1]。
- **scales** - `T3` (可选):
    - Shape: N[4]
    - 各维度缩放因子 [batch_scale, channel_scale, height_scale, width_scale]，与 sizes 互斥。linear、nearest 模式下仅支持整数倍缩放，且 batch/channel 维度 scale 必须为 1，H/W 维度 scale <= 8。
- **sizes** - `T4` (可选):
    - Shape: N[4]
    - 目标输出尺寸 [batch, channel, height, width]，与 scales 互斥。linear、nearest 模式下仅支持整数倍缩放，且 batch/channel 维度不变，H/W 维度倍数 <= 8。

输出

- **Y** - `T1`:
    - Shape: NCHW[batch, channel, out_height, out_width]
    - 量化: per-layer

属性

- **mode - string**:
    - linear、nearest 模式内部通过卷积模拟实现。
- **coordinate_transformation_mode - string**:
    - 不支持 tf_crop_and_resize。
- **cubic_coeff_a - float**:
    - cubic 模式系数，当前仅支持默认值 -0.75。
- **exclude_outside - int**:
    - 不支持排除外部点。
- **extrapolation_value - float**:
    - 不支持外推值设置。
- **nearest_mode - string**:
    - nearest 模式下的取整方式。
- **antialias - int**:
    - 不支持抗锯齿。
- **keep_aspect_ratio_policy - string**:
    - 仅支持默认拉伸模式。
- **axes - int[axis_0, axis_1, axis_2, axis_3]**:
    - 指定缩放维度，当前实现未对该属性做额外约束。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T3** in (`tensor<float32>`): **T4** in (`tensor<int64>`):

---

exWindow

exWindow来源于 https://github.com/microsoft/Swin-Transformer/blob/main/models/swin_transformer.py 的window_partition和window_reverse操作。原论文中输入特征被划分为多个不重叠的窗口，每个窗口的大小为window_size × window_size。要求输入和输出特征都为4维shape，在高和宽方向上都进行划分。

公式

```
B, C, H, W = X.shape

if mode == "partition":
    x = X.reshape(B, C, num_windows[0], window_sizes[0],
                  num_windows[1], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])

elif mode == "reverse":
    x = X.reshape(B, C, num_windows[0], num_windows[1],
                  window_sizes[0], window_sizes[1])
    y = np.transpose(x, (0, 1, 2, 4, 3, 5))
    Y = y.reshape(B, C,
                  num_windows[0] * window_sizes[0],
                  num_windows[1] * window_sizes[1])

elif mode == "partition_num_first":
    x = X.reshape(B, C, window_sizes[0], num_windows[0],
                  window_sizes[1], num_windows[1])
    y = np.transpose(x, (0, 1, 3, 5, 2, 4))
    Y = y.reshape(B, C,
                  num_windows[0] * num_windows[1],
                  window_sizes[0] * window_sizes[1])
```

算子层级

RKNN3扩展算子

输入

- **X** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - partition 要求 height = num_h * win_h，width = num_w * win_w。
    reverse 要求 height = num_h * num_w，width = win_h * win_w。

partition_num_first 要求 height = win_h * num_h，width = win_w * num_w。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, out_height, out_width]
  - 量化: per-layer
  - partition 和 partition_num_first 输出 out_height = num_h * num_w，out_width = win_h * win_w。
    reverse 输出 out_height = num_h * win_h，out_width = num_w * win_w。

属性

- **mode - string**:
  - partition 参考 window_partition 实现。
    reverse 参考 window_reverse 实现。
    partition_num_first 与 partition 的区别在于，partition_num_first 的 H/W 按照 num_windows 进行划分，而 partition 的 H/W 按照 window_sizes 进行划分。
- **num_windows - int[num_h, num_w]**:
  - num_h 范围: [1, 8192]
  - num_w 范围: [1, 8192]
  - 2维向量,窗口的数量。定义了输入特征的高度和宽度被划分为多少个窗口。
- **window_sizes - int[win_h, win_w]**:
  - win_h 范围: [1, 8192]
  - win_w 范围: [1, 8192]
  - 2维向量,窗口的大小。定义了每个窗口的高度和宽度。窗口内的注意力计算和特征提取都是基于这个大小。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

索引与散布

Gather

沿指定轴从输入张量中根据索引收集元素。

公式

```
output = Gather(input, indices, axis)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::embedding`, `aten::gather`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **indices** - `T2`:
  - Shape: INDICES[batch, index]

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **axis - int**:
  - 沿此轴收集元素。

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`tensor<int32>`):

GatherElements

沿指定轴按索引从输入张量中收集元素，输出 shape 与 indices 相同。

公式

```
output = GatherElements(data, indices, axis)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::gather`

输入

- **data** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **indices** - `T2`:
  - Shape: NCHW[batch, channel, height, width]
  - 支持常量和非常量索引。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **axis - int**:
  - 沿此轴收集元素，负值会归一化到有效范围。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<int32>`): **T2** in (`tensor<int32>`):

GatherND

根据 indices 从输入张量中收集切片，组合成新的输出张量。

公式

```
output = GatherND(data, indices, batch_dims)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::gather_nd`

输入

- **data** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

- **indices** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

输出

- **output** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **batch_dims - int**:
  - 指定从开头的多少个维度作为 batch 维度，这些维度上不做 gather 操作。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>`):

ScatterND

按 indices 将 updates 散布到 data 张量的指定位置，生成输出张量，支持 reduction 归约模式。

公式

```
output = ScatterND(data, indices, updates, reduction)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::scatter_nd`

输入

- **data** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **indices** - `T2` :
  - Shape: INDICES[num_index, index_dim]
- **updates** - `T1` :
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **reduction - string**:
  - 指定散布时的归约方式。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`tensor<int32>` , `tensor<int64>`):

归一化

BatchNormalization

对输入张量进行批量归一化计算（参考 https://arxiv.org/abs/1502.03167）。

公式

```
output = (input - mean) / sqrt(variance + epsilon)
```

算子层级

标准ONNX算子

PYTORCH 来源

`aten::batch_norm`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **scale** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel
- **B** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel

- **mean** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel
  - 推理时与 var 一起被折叠进 scale/B，运行期不单独参与计算。
- **var** - `T2`:
  - Shape: C[channel]
  - 量化: per-channel
  - 推理时与 mean 一起被折叠进 scale/B，运行期不单独参与计算。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer

属性

- **epsilon - float**:
  - 除以标准差时加上防止除 0 的小实数，默认 1e-5。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`): **T2** in (`const_tensor<float16>`):

BatchNormRelu

BatchNormalization 与 Relu 融合计算。

公式

```
output = Relu(BatchNormalization(input))
```

继承关系

继承自 BatchNormalization。

算子层级

融合算子

PYTORCH 来源

```
aten::batch_norm
```

---

exNorm

执行扩展归一化计算，并可选应用 gamma 缩放和 beta 偏移。

公式

```
x = X
weight = W if W is not None else 1
bias = B if B is not None else 0

if mode == "layer":
    x_centered = x - np.mean(x, axis=axes, keepdims=True)
    y = x_centered * fast_inv_sqrt(
        np.mean(x_centered * x_centered, axis=axes, keepdims=True) + eps)

elif mode == "mean_var":
    mean = np.mean(x, axis=axes, keepdims=True)
    var = np.mean(x * x, axis=axes, keepdims=True) - mean * mean
    y = (x - mean) * fast_inv_sqrt(var + eps)

elif mode == "rms":
    y = x * fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "pixel":
    y = x * fast_inv_sqrt(
        np.sum(x * x, axis=axes, keepdims=True) + eps)

elif mode == "l1":
    y = x / np.maximum(
        np.sum(np.abs(x), axis=axes, keepdims=True), eps)

elif mode == "l2":
    y = x / np.maximum(
        fast_sqrt(np.sum(x * x, axis=axes, keepdims=True)), eps)

elif mode == "rms_reciprocal":
    y = fast_inv_sqrt(
        np.mean(x * x, axis=axes, keepdims=True) + eps)

elif mode == "gaussian_topk":
    mean = np.mean(x, axis=axes, keepdims=True)
    std = fast_sqrt(np.mean(x * x, axis=axes, keepdims=True) - mean * mean)
    cutoff_x = mean + std * std_multipler
    y = np.clip(x - cutoff_x, a_min=0, a_max=None).astype(x.dtype)

Y = y * weight + bias
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::layer_norm`, `aten::rms_norm`

输入

- **X** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
- **W** - `T1` (可选):
  - Shape: NCHW[batch, channel, height, width], [1]
  - 量化: per-layer
  - gamma 缩放参数，支持按 layer、channel 或与输出兼容的广播形状。
- **B** - `T1` (可选):
  - Shape: NCHW[batch, channel, height, width], [1]
  - 量化: per-layer
  - beta 偏移参数，支持按 layer、channel 或与输出兼容的广播形状。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - mode 为 rms_reciprocal 时，axes 指定的维度保持 keepdims 语义并输出为 1。

属性

- **mode - string**:
  - 可选值范围: layer, mean_var, rms, pixel, l1, l2, rms_reciprocal, gaussian_topk。
- **axes - int[axis0, axis1, axis2]**:
  - 默认值为 [1]。
    shape 推导支持负轴；NPU 子图实现主要支持 [1] 和 [1, 2, 3]。
- **eps - float**:
  - 默认值为 1e-5，用于避免除零。
- **std_multipler - float**:
  - 仅 gaussian_topk 使用，默认值为 1.0。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exSoftmax

对输入沿指定轴做 softmax 归一化，输出与输入形状相同。

公式

`exSoftmax(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::softmax`

输入

- **input** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 受限于 transpose 的规格限制。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 做 softmax 的轴，1 为 channel 方向，3 为 width 方向。仅支持 axis 1 和 3 是 RKNN 平台限制，ONNX 标准 axis 可为任意有效轴。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

---

exSoftmaxMask

带 mask 的 Softmax 计算，先按 mask_value 将 mask 位置置为 -inf，再做 Softmax。

公式

`input(x, mask_t, mask_value) = x + (mask_t - 1) * inf, if mask_value = 0`
`x + mask_t * (-inf), if mask_value = 1`
`exSoftmaxMask(input, axis) = exp(input - max(input, axis)) / sum_axis(exp(input - max(input, axis)))`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::softmax`

输入

- **input** - `T1` (必选):
  - Shape: NCHW[batch, channel, height, width]
  - 受限于 transpose 的规格限制。
- **mask** - `T1` (可选):
  - Shape: NCHW[1, channel, height, width]
  - channel 范围: [1, 8192]
  - width 范围: [1, 8192]
  - axis=1 时 width 无限制；axis=3/-1 时 width 取值 [1, 8192]。

输出

- **output** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

属性

- **axis - int**:
  - 做 softmax 的轴，1 为 channel 方向，3 为 width 方向。仅支持 axis 1 和 3 是 RKNN 平台限制，ONNX 标准 axis 可为任意有效轴。
- **mask_value - int**:
  - 0 表示保留位置，1 表示需要 mask 的位置。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>`):

矩阵运算

exMatMul

矩阵乘法操作

公式

`Y = A · B + C, C 为 bias（bias 可选）`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::matmul`, `aten::bmm`, `aten::mm`

输入

- **A** - `T1` (必选):
  - Shape: NK1N[batch, k, 1, n]
  - 量化: per-layer、per-channel、group32/64/128
- **B** - `T1` (必选):
  - Shape: NK1M[batch, k, 1, m]
  - 量化: per-layer
- **C** - `T1, T2` (可选):
  - Shape: NM1M[batch, n, 1, m]
  - 量化: per-layer
  - 支持 Numpy 样式广播，广播约束同 Expand 算子。

输出

- **Y** - `T1`:
  - Shape: NM1M[batch, n, 1, m]
  - 量化: per-layer

数据类型约束

T1 in (`tensor<int8>`, `tensor<float16>`): **T2** in (`const_tensor<float16>`):

---

exMatMulActivation

exMatMul 与通用 Activation 融合计算，激活类型由 Activation 子算子的类型决定。

公式

`output = Activation(exMatMul(input, weight, bias))`

继承关系

继承自 exMatMul。

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::matmul`, `aten::bmm`, `aten::mm`

LLM 扩展算子

exAttention

支持 FullAttention/SlidingAttention 两种模式的通用注意力子图算子。

公式

```
output = Matmul(Softmax(Matmul(rope(query, cos_pos_emb, sin_pos_emb), rope(key, cos_pos_emb, sin_pos_emb)) + attention_mask), value)
output = Matmul(Softmax(Matmul(query, key) + attention_mask), value)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::multi_head_attention_forward`

输入

- **query** - `T1` (必选):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
- **key** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **attention_mask** - `T1` (必选):
  - Shape: NCHW[1, seq_len, 1, seq_len]
- **cos_pos_emb** - `T1` (可选):
  - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **sin_pos_emb** - `T1` (可选):

    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
- **Th** - `T2` (必选):
    - Shape: N[1]
    - history token len，历史已经推理过的 token 数。
- **Tc** - `T2` (必选):
    - Shape: N[1]
    - current token len，本次推理桶内的有效的 token 数。
- **Ts** - `T2` (必选):
    - Shape: N[1]
    - system prompt token len, 对齐后的系统提示词长度，[182X, 3572] 均需要对齐到 32 的整数倍。
- **Tsr** - `T2` (必选):
    - Shape: N[1]
    - real system prompt token len, 实际的系统提示词长度，小于等于 Ts。

输出

- **output** - `T1` :
    - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]

属性

- **attention_type - string**:
    - 分别表示全局注意力和滑动窗口注意力两种模式，可选值为[FullAttention, SlidingAttention]，默认 FullAttention。
- **rope - int**:
    - 是否使用 RoPE 位置编码，0 表示不使用，1 表示使用，默认 0。
- **kvcache_store_method - string**:
    - 表示 KV cache 的存储方式，Normal 表示直接存储 float16 数据，GroupQuant 表示按组量化存储，可选值为[Normal, GroupQuant]，默认 Normal。
- **kvcache_dtype - string**:
    - 表示 KV cache 的数据类型，可选值为[Float16, Int4_to_F8, Int4_to_F16]，Float16 表示使用 float16 存储，Int4_to_F8 表示使用 4-bit 整数量化存储并在计算时转换为 8-bit 浮点数，Int4_to_F16 表示使用 4-bit 整数量化存储并在计算时转换为 16-bit 浮点数，默认 Float16。
- **kvcache_buffer_len - int**:
    - KV cache 的最大长度，单位为 token，需要小于模型的 max_position_embeddings。
- **kvcache_group_size - int**:
    - 当 kvcache_store_method 为 GroupQuant 时有效，表示分组量化的组大小；当 kvcache_store_method 为 Normal 时，默认设置为 32。

- **kvcache_residual_depth - int**:
  - 当 kvcache_store_method 为 GroupQuant 时有效，表示存满该数值 token 后才触发分组量化；当 kvcache_store_method 为 Normal 时，默认设置为 32。
- **sliding_window_size - int**:
  - 当 attention_type 为 SlidingAttention 时有效，表示滑动窗口的大小；当 attention_type 为 FullAttention 时，默认设置为 -1。

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

---

exPagedAttention

使用分页 KV cache 的注意力子图算子，支持 FullAttention、SlidingAttention 和多请求 Decode。

公式

```
Q = rope(query, cos_pos_emb, sin_pos_emb) if rope == 1 else query
K_current = rope(key, cos_pos_emb, sin_pos_emb) if rope == 1 else key
K_all, V_all = concat(历史有效 KV cache, 本轮有效 K_current/value)
logits[h,q,k] = scale * dot(Q[h,q], K_all[kv_head(h),k]) + mask[q,k]
P[h,q,k] = exp(logits[h,q,k]) / (sum_j exp(logits[h,q,j]) + (exp(sinks[h]) if
has_sink == 1 else 0))
output[h,q] = sum_k P[h,q,k] * V_all[kv_head(h),k]
```

算子层级

RKNN3扩展算子

输入

- **query** - `T1` (必选):
  - Shape: NCHW[num_attention_heads, head_dim, 1, seq_len]
  - num_attention_heads 必须是 num_key_value_heads 的整数倍，支持 MHA、MQA 和 GQA。
- **key** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, head_dim, 1, seq_len]
- **value** - `T1` (必选):
  - Shape: NCHW[num_key_value_heads, value_head_dim, 1, seq_len]
- **attention_mask** - `T1` (必选):
  - Shape: NCHW[1, seq_len, 1, seq_len]
  - 本轮 token 之间的加性掩码；历史 cache 的有效区间和 SlidingAttention 的窗口掩码由内部构图处理。
- **sinks** - `T3` (可选):

    - Shape: N[num_attention_heads]
    - has_sink=1 时紧跟 attention_mask，必须为 FP32 initializer，并按当前导入器要求在 graph input 中声明；每个 query head 一个 logit，仅加入 Softmax 分母，不参与 scale、RoPE 或 mask，不写入 KV cache，后续输入顺延。
- **cos_pos_emb** - `T1` (可选):
    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
    - rope=1 时必须提供，位于 attention_mask 或可选 sinks 之后。
- **sin_pos_emb** - `T1` (可选):
    - Shape: NCHW[1, head_dim, 1, max_position_embeddings]
    - rope=1 时必须提供，紧跟 cos_pos_emb。
- **control_variables** - `T2` (必选):
    - Shape: NC[request_num, 9]
    - 每个请求一行，必须保持九列顺序为 [Th, Tc, Ts, Tsr, longest_surviving_id, store_begin_id, Tr, Ti, request_masks]。
      Th 为本轮开始前的历史 token 数；Tc 为本轮有效 token 数，不超过 bucket_size，Decode 时为 1。
      Ts 为对齐后的系统提示词长度，需对齐到 32 的整数倍且小于 kvcache_buffer_len；Tsr 为实际系统提示词长度，满足 0 <= Tsr <= Ts。
      longest_surviving_id 为主 cache 中最早仍有效的 token 存储位置，空 cache 为 -1；
      store_begin_id 为主 cache 下一次写入的位置。
      Tr 为 residual cache 中的有效 token 数；Ti 为上轮结束的逻辑 token 位置，用于与 Th 比较并处理回退。
      request_masks 在多请求时控制本请求的 Attention 和 KVStore，0 表示跳过，非 0 表示执行；单请求时按 1 处理。
      longest_surviving_id、store_begin_id、Tr、Ti 是可回写状态，后续推理必须与对应 KV cache 一起延续；Th/Tc/Ts/Tsr 由调用方提供。
      新请求可使用 [0, Tc, Ts, Tsr, -1, 0, 0, 0, 1] 初始化，Th=0 时内部重置 cache 状态。

输出

- **output** - `T1` :
    - Shape: NCHW[num_attention_heads, value_head_dim, 1, seq_len]

属性

- **attention_type - string**:
    - 显式指定全局注意力或滑动窗口注意力模式。
- **scale - float**:
    - QK logits 的缩放系数，默认 1.0；需要标准缩放点积注意力时由调用方设置为 1/sqrt(head_dim)。
- **has_sink - int**:
    - 默认 0；1 表示存在 sinks 常量输入，FullAttention 和 SlidingAttention 均可使用。

- **rope - int**:
  - 默认 0；1 表示在内部对 query/key 应用 RoPE，需要同时提供 cos_pos_emb 和 sin_pos_emb；外置 RoPE 时设置为 0。
- **kvcache_store_method - string**:
  - 表示 KV cache 的数据类型，Float16 表示使用 float16 存储，Int4_to_F8 表示使用 4-bit 整数量化存储并在计算时转换为 8-bit 浮点数，Int4_to_F16 表示使用 4-bit 整数量化存储并在计算时转换为 16-bit 浮点数，默认 Float16。
- **kvcache_dtype - string**:
  - Normal 使用 Float16；其余类型用于 GroupQuant，后缀 F8/F16 表示计算路径使用的精度；未指定时使用编译配置。
- **kvcache_buffer_len - int**:
  - 每个请求主 KV cache 的最大 token 容量，必须为正数；未指定或设置为 -1 时使用编译配置。
- **tokens_per_page - int**:
  - 每页 KV cache 的 token 数，默认 512，必须为正数；大于 kvcache_buffer_len 时截断为该长度。FullAttention 按此大小分页；SlidingAttention 使用覆盖整个 kvcache_buffer_len 的单页。
- **kvcache_group_size - int**:
  - 当 kvcache_store_method 为 GroupQuant 时有效，表示分组量化的组大小；当 kvcache_store_method 为 Normal 时，默认设置为 32。
- **kvcache_residual_depth - int**:
  - 当 kvcache_store_method 为 GroupQuant 时有效，表示存满该数值 token 后才触发分组量化；当 kvcache_store_method 为 Normal 时，默认设置为 32。
- **request_num - int**:
  - 默认 1；实际请求数由 control_variables 的第 0 维静态确定，属性应与该维一致。
- **sliding_window_size - int**:
  - SlidingAttention 的窗口大小；FullAttention 时不生效，默认 -1。

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`): **T3** in (`tensor<float>`):

---

exSDPAttention

执行 scaled dot-product attention，可选叠加 mask。

公式

```
q = Query
k = Key
v = Value

score = MatMul(k, q) * scale
if Mask is not None:
    score = score + Mask

prob = Softmax(score, axis=1)
Y = MatMul(v, prob)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::scaled_dot_product_attention`

输入

- **Query** - `T1` (必选):
  - Shape: NCHW[batch, query_length, head, query_channel]
  - 量化: per-layer
  - q 矩阵输入，对应代码中的 query。
- **Key** - `T1` (必选):
  - Shape: NCHW[batch, key_length, head, key_channel]
  - 量化: per-layer
  - k 矩阵输入，对应代码中的 key，参与 qk_matmul。
- **Value** - `T1` (必选):
  - Shape: NCHW[batch, value_length, head, value_channel]
  - 量化: per-layer
  - v 矩阵输入。
- **Mask** - `T1` (可选):
  - Shape: NCHW[batch, key_channel, head, query_channel]
  - 量化: per-layer
  - attention mask，存在时在 softmax 前与 qk_matmul 结果相加，形状需要可广播到 qk_matmul 输出。

输出

- **SDPAOutput** - `T1`:

- Shape: NCHW[batch, query_length, head, value_channel]
- 量化: per-layer
- scaled dot-product attention 的最终输出。

属性

- **scale - float**:
  - 默认值为 1.0，作用在 qk_matmul 结果上。
- **QMatmul - string**:
  - qk_matmul 输出量化参数描述。
- **QAdd - string**:
  - mask add 输出量化参数描述，仅 Mask 存在时使用。
- **QSoftmax - string**:
  - softmax 输出量化参数描述。

数据类型约束

T1 in (`tensor<int8>` , `tensor<float16>` , `tensor<bfloat16>`):

---

exLinearAttention

Chunk Gated Delta Rule 线性注意力子图算子，按 chunk 分块计算 query/key/value 的递归线性注意力。

公式

```
output = LinearAttention(query, key, value, g, beta, initial_state)
```

算子层级

RKNN3扩展算子

输入

- **query** - `T1` (必选):
  - Shape: HD1S[H, D, 1, S]
  - 量化: per-layer
  - H 为 num_heads，D 为 head_dim，S 为 seq_len。
- **key** - `T1` (必选):
  - Shape: HD1S[H, D, 1, S]
  - 量化: per-layer
- **value** - `T1` (必选):

    - Shape: HD1S[H, D, 1, S]
    - 量化: per-layer
- **g** - `T1` (必选):
    - Shape: 1H1S[1, H, 1, S]
    - 量化: per-layer
    - gate 门控因子。
- **beta** - `T1` (必选):
    - Shape: 1H1S[1, H, 1, S]
    - 量化: per-layer
- **Tc** - `T2` (必选):
    - Shape: S[seq_len]
    - 有效 seq_len 标量。当 Tc < seq_len 时，对 query/key/value 在 seq 维做左侧 zero padding。
- **initial_state** - `T1` (可选):
    - Shape: HDD[H, D, 1, D]
    - 量化: per-layer
    - 可选的初始 recurrent state。若前端提供则直接接入。

输出

- **output** - `T1`:
    - Shape: HD1S[H, D, 1, S]
    - 量化: per-layer
- **final_state** - `T1`:
    - Shape: HDD[H, D, 1, D]
    - 量化: per-layer
    - 仅当 OutputFinalState 为 true 时输出。

属性

- **ChunkSize - int**:
    - chunk 分块大小，默认 32。seq_len 必须是 ChunkSize 的整数倍（由调用方负责 padding）。
- **UseQkL2Norm - bool**:
    - 是否对 query/key 做 L2 归一化，默认 false。
- **OutputFinalState - bool**:
    - 是否输出最终 recurrent state，默认 false。

数据类型约束

T1 in (`tensor<float16>` , `tensor<bfloat16>`): **T2** in (`tensor<int32>`):

说明

子图算子，实际实现由 createSubGraph 在 IR 层展开，emit 仅返回成功。
batch 维固定为 1。seq_len 需在进入子图前 pad 到 ChunkSize 整数倍。

---

exRoPE

支持 RoPE 位置编码的通用子图算子。

公式

```
output = input * cos_pos_emb + rotate_half(input) * sin_pos_emb
```

算子层级

RKNN3扩展算子

输入

- **input** - `T1` (必选):
  - Shape: NCHW[batch_size, channel, 1, seq_len]
- **cos_pos_emb** - `T1` (必选):
  - Shape: NCHW[batch_size, channel, 1, position_embeddings]
  - position_embeddings == 1 or
    seq_len <= position_embeddings <= max_position_embeddings。
- **sin_pos_emb** - `T1` (必选):
  - Shape: NCHW[batch_size, channel, 1, position_embeddings]
  - position_embeddings == 1 or
    seq_len <= position_embeddings <= max_position_embeddings。
- **Th** - `T2` (可选):
  - Shape: N[1]
  - history token len，历史已经推理过的 token 数。
- **Tc** - `T2` (可选):
  - Shape: N[1]
  - current token len，本次推理桶内的有效的 token 数。
- **Ts** - `T2` (可选):
  - Shape: N[1]
  - system prompt token len, 对齐后的系统提示词长度，[182X, 3572] 均需要对齐到 32 的整数倍。

- **Tsr** - `T2` (可选):
  - Shape: N[1]
  - real system prompt token len, 实际的系统提示词长度，小于等于 Ts。

输出

- **output** - `T1`:
  - Shape: NCHW[batch_size, channel, 1, seq_len]

数据类型约束

T1 in (`tensor<float16>`): **T2** in (`tensor<int32>`):

说明

当 exRoPE 用于 llm 模型，并传入全量的 cos/sin_pos_emb 时(position embeddings > seq_len)，会存在 Th/Tc/Ts/Tsr 等输入，exRoPE 内部会根据 Th/Tc/Ts/Tsr 计算出实际需要使用的 cos/sin_pos_emb 的长度，从而支持动态序列长度的推理。其他情况 seq_len == position_embeddings，Th/Tc/Ts/Tsr 不存在。

---

exSum

多输入加权求和，常用于 LoRA 多分支结果合并。

公式

`Y = Sum(X_i * scale_i) for i in 0..N-1`

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::sum`

输入

- **X** - `T1`:
  - Shape: NCHW[batch, channel, height, width]
  - 量化: per-layer
  - 可变长度输入列表，至少 1 个，所有输入形状必须一致。

输出

- **Y** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

- 量化: per-layer

属性

- **scales - float[scale_1, ..., scale_N-1]**:
  - 每个额外输入（除第一个外）的加权系数，数量为输入个数 - 1。

数据类型约束

T1 in (`tensor<float16>`):

类型转换

Cast

将输入张量的元素逐个转换为指定目标数据类型。

公式

`output = Cast(input, to)`

算子层级

标准ONNX算子

PYTORCH 来源

`aten::to`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

输出

- **output** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

属性

- **to - int**:
  - 目标数据类型，由 ONNX to_type 映射而来，支持 bool/int8/uint8/int16/int32/float16/float32。

数据类型约束

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

---

exDataConvert

将输入张量的数据类型转换为指定目标类型。

公式

```
output = exDataConvert(input, input_type, output_type)
```

算子层级

RKNN3扩展算子

PYTORCH 来源

`aten::to`

输入

- **input** - `T1` :
  - Shape: NCHW[batch, channel, height, width]

输出

- **output** - `T2` :
  - Shape: NCHW[batch, channel, height, width]

属性

- **input_type - int**:
  - 输入数据类型。
- **output_type - int**:
  - 输出数据类型。

数据类型约束

T1 in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`): **T2** in (`tensor<bool>` , `tensor<int8>` , `tensor<uint8>` , `tensor<int16>` , `tensor<int32>` , `tensor<float16>` , `tensor<float32>`):

输入输出处理

exInputProc

执行输入数据的预处理。

算子层级

RKNN3扩展算子

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

输出

- **B** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`): **T2** in (`tensor<float16>` , `tensor<int8>`):

---

exOutputProc

执行输出数据的后处理。

算子层级

RKNN3扩展算子

输入

- **A** - `T1`:
  - Shape: NCHW[batch, channel, height, width]

输出

- **B** - `T2`:
  - Shape: NCHW[batch, channel, height, width]

数据类型约束

T1 in (`tensor<float16>` , `tensor<int8>`): **T2** in (`tensor<float32>` , `tensor<float16>` , `tensor<bool>` , `tensor<uint8>`):